API Security Testing Framework

Internship Master Report

Based on OWASP API Security Top 10 (2023)

Lab targets: OWASP crAPI, OWASP Juice Shop | Primary tool: Burp Suite

Table of Contents

Kickoff & Environment Setup	3
OWASP API Top 10 (2023) — Part 1	4
OWASP API Top 10 (2023) — Part 2	5
OWASP API Top 10 (2023) — Part 3 + Week 1 Wrap	6
API Discovery	7
Authentication & JWT Testing	8
BOLA / IDOR Testing	10
Broken Function Level Authorization (BFLA) + Excessive Data Exposure / Mass Assignment	13
Rate Limiting + SSRF + Security Misconfiguration + SQL Injection	16
Injection Testing — SQL Injection & NoSQL Injection on Community Posts	21
Test Case Padding — BOLA, IDOR, SQL Injection & NoSQL Injection Edge Cases	31
VAPT Report	36
API Security Checklist	51
Remediation Tracker	54

Kickoff & Environment Setup

What I Learned

API (Application Programming Interface): This is basically how two programs talk to each other over a network — like a mobile app asking a server to send back your account balance. Pretty much every app relies on APIs behind the scenes, even when we don't notice it.

API Security Testing: Trying to break an API on purpose (safely and legally) to find spots where it doesn't enforce its own rules properly, like access control or request limits — basically finding the weak points before a real attacker does.

OWASP (Open Web Application Security Project): A nonprofit that puts out free security standards for the industry. They're also the ones behind crAPI and Juice Shop, the two apps I'll be practicing on.

crAPI (Completely Ridiculous API): An API that OWASP built on purpose to be full of vulnerabilities, made specifically so people can practice attacking it legally instead of testing on a real app.

Docker: Packages up an app with everything it needs — code, dependencies, config — into one "container" that runs the same way no matter what machine you put it on.

Docker Compose: Lets you start a bunch of linked containers together (like an app plus its database) from one config file, instead of starting each one by hand.

Burp Suite / Proxy Interception: Sits between my browser and whatever app I'm testing so I can see and edit every request before it reaches the server. This is the main tool I'll be using for basically the whole internship.

FoxyProxy: A Firefox extension that lets me quickly flip Burp's proxy on and off instead of digging into Firefox's network settings every single time.

Tools Used

Kali Linux, Docker, Docker Compose, OWASP crAPI, OWASP Juice Shop, Burp Suite Community Edition, FoxyProxy, Oracle VirtualBox

What I Did

Installed Docker and Docker Compose on Kali Linux, turned on the Docker service, and added my user account to the docker group so I don't have to sudo everything. Ran the official hello-world test image to confirm it actually worked.

Cloned the OWASP crAPI repo from GitHub and found its Docker Compose config inside the deploy/docker folder. Pulled all 10 required containers (web frontend, three backend microservices, two databases, and a few supporting services) and brought the whole stack up with docker compose up -d. Checked that crAPI's login page loaded fine at http://localhost:8888.

Burp Suite Community Edition was already installed on Kali. Pointed Firefox at Burp's proxy listener (127.0.0.1:8080) using the FoxyProxy extension.

Ran into a Firefox setting (network.proxy.allow_hijacking_localhost) that was quietly blocking extension-based proxies from routing localhost traffic to Burp. Turned it on through about:config and after that crAPI's traffic started showing up properly in Burp's HTTP History.

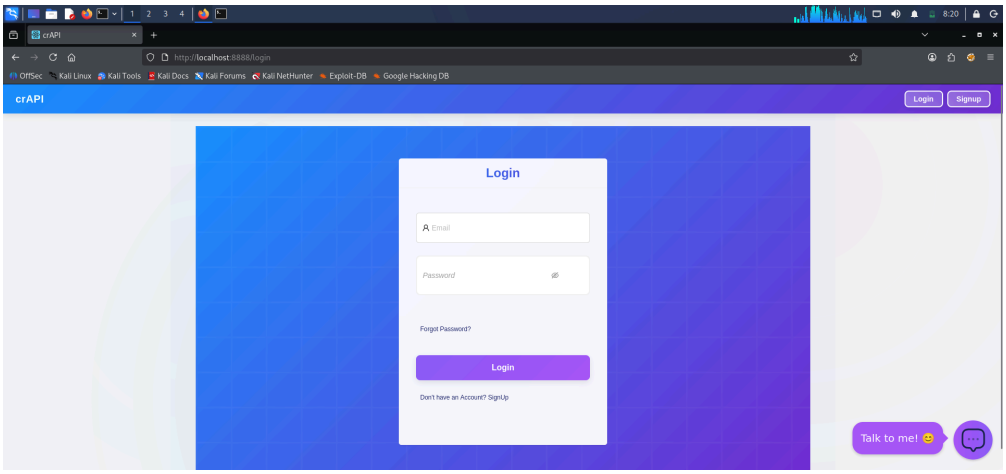


Figure 1: *OWASP crAPI login page successfully running at localhost:8888.*

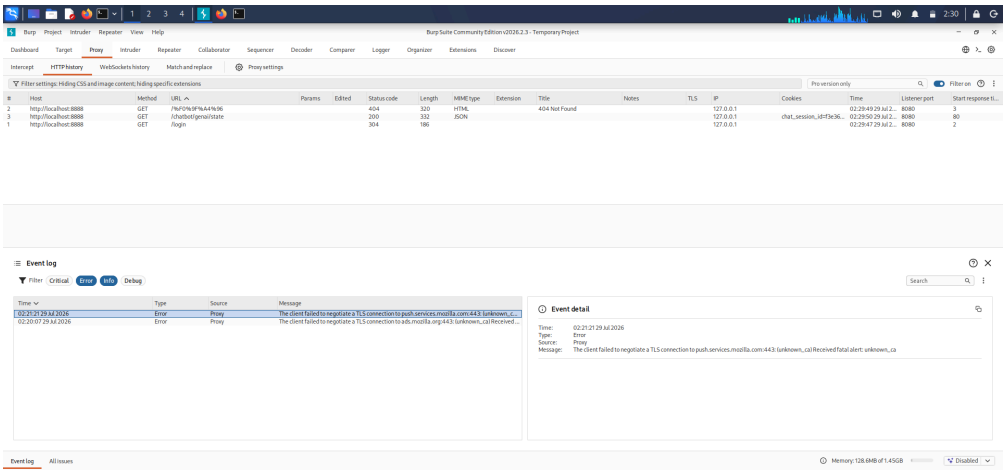


Figure 2: *Burp Suite HTTP History showing intercepted crAPI requests, confirming the proxy is correctly capturing local traffic.*

Findings

N/A — setup day, no security testing performed.

Test Cases Added

0 (running total: 0 / 100+)

Challenges

The Kali VM only had 1.9 GB of RAM allocated at first, which wasn't enough to run crAPI's 10 containers alongside Burp and Firefox at the same time — it froze up under memory pressure (I could see it hitting swap). Fixed it by bumping the VM up to 6 GB of RAM and 4 CPU cores in VirtualBox.

After bumping up the resources, the VM wouldn't even power on and threw a VERR_UNRESOLVED_ERROR. Turned out Windows' “Virtual Machine Platform” feature was conflicting with VirtualBox's virtualization mode. Disabled that feature in Windows Features and restarted the host, which fixed it.

Burp wasn't showing any crAPI traffic in HTTP History even though FoxyProxy looked active. Turned out to be the same Firefox localhost-proxy setting mentioned above (see What I Did) — fixed it the same way.

OWASP API Top 10 (2023) — Part 1

What I Learned

API1 – Broken Object Level Authorization (BOLA): This is when the server checks that an object ID exists, but doesn't check whether that ID actually belongs to whoever's asking for it. So I could change an ID in a request (like a vehicle ID or document ID) and pull up someone else's data instead of my own. Also called IDOR (Insecure Direct Object Reference).

API2 – Broken Authentication: Happens when the API can't properly confirm someone really is who they claim to be. Usually caused by things like no lockout after repeated failed logins, weak password rules, or accepting weak/forged JWTs (JSON Web Tokens — signed strings that prove you're logged in), such as tokens that aren't actually signed or don't check expiry. It's a nasty category because once it's broken, the app can no longer tell an attacker's actions apart from the real user's.

API3 – Broken Object Property Level Authorization: This one has two sides. Excessive Data Exposure is when the API sends back more fields than the app actually needs — like accidentally including a hashed password or an isAdmin flag in a response even though the UI never displays it. Mass Assignment goes the other direction: the API accepts more fields from the client than it should, so a user could sneak in a field they're not supposed to touch, like adding “total_stay_price” to a booking request.

Tools Used

Web browser, notes document

What I Did

Read through the official OWASP pages for API1, API2, and API3 — went over the vulnerability description, example attacks, and prevention guidance for each, and wrote a paragraph in my own words per category. Also spent some time just poking around crAPI as a normal user to get familiar with it before Week 2 — made two test accounts and clicked through the main features (profile, vehicles, marketplace) while watching Burp's HTTP History to see how requests, IDs, and auth tokens actually look in real traffic.

Findings

N/A — theory/notes day; crAPI exploration was for orientation only, no testing performed.

Test Cases Added

0 (running total: 0 / 100+)

Challenges

No major challenges. Needed a second pass to make sure API3's two halves (Excessive Data Exposure vs. Mass Assignment) were both clear, since they kind of blend together into one idea at first glance.

OWASP API Top 10 (2023) — Part 2

What I Learned

API4 – Unrestricted Resource Consumption: Happens when an API doesn't put any limit on how many requests a user can send, how much data they can send, or how large a file they can upload. Without limits, an attacker can flood the server with requests, which slows the API down or takes it offline for real users — and can even run up server/cloud costs since a lot of APIs are billed per request or per compute used.

API5 – Broken Function Level Authorization (BFLA): Happens when an API checks that a user is logged in (authentication) but doesn't check whether that specific user's role is actually allowed to call a particular function (authorization). So a regular customer's token might still be able to call an admin-only endpoint — like one that deletes another user's account — simply because the server never checked the caller's privilege level before running it.

API6 – Unrestricted Access to Sensitive Business Flows: This isn't about broken code — it's a completely normal, working feature being abused at scale, usually through automation. Like a bot script buying up every concert ticket the second they go on sale, or submitting thousands of fake signups to claim a promo. Each individual action looks totally legitimate to the API; it's the automated scale that actually damages the business.

Tools Used

Web browser, notes document

What I Did

Read the official OWASP pages for API4, API5, and API6, going over the vulnerability description, example attacks, and prevention guidance for each. Wrote a paragraph in my own words per category, covering Unrestricted Resource Consumption, Broken Function Level Authorization, and Unrestricted Access to Sensitive Business Flows.

Findings

N/A — theory/notes day, no hands-on testing performed.

Test Cases Added

0 (running total: 0 / 100+)

Challenges

No major challenges.

OWASP API Top 10 (2023) — Part 3 + Week 1 Wrap

What I Learned

API7 – SSRF (Server-Side Request Forgery): SSRF is when a feature that fetches a URL on your behalf (like an image-from-URL uploader) gets tricked into fetching an internal address instead — letting an attacker use the server's own network access to reach internal systems they shouldn't be able to touch directly.

API8 – Security Misconfiguration: Covers basic security setup mistakes — stuff like default or leaked passwords still being active, error messages that are too verbose and leak internal details, and missing security headers that are supposed to protect the app.

API9 – Improper Inventory Management: Companies often keep old API versions running in the background after releasing a newer one, and since nobody's actively tracking or monitoring the old one anymore, it never gets patched — making it an easy, overlooked way in for attackers.

API10 – Unsafe Consumption of APIs: This is when an API blindly trusts data coming from a third-party API without validating it, just assuming it's safe because it came from a “trusted” partner — which can let malicious or bad data through and be used to steal information or gain access.

Tools Used

Web browser, notes document

What I Did

Read the official OWASP pages for API7, API8, API9, and API10, going over the vulnerability description, example attacks, and prevention guidance for each, and wrote a paragraph in my own words per category. Also started building the skeleton API Security Checklist doc — just the 10 OWASP category headers for now, empty, to be filled in as I actually test things over the coming weeks.

Findings

N/A — theory/notes day, no hands-on testing performed.

Test Cases Added

0 (running total: 0 / 100+)

Challenges

Needed a few passes on API9 and API10 to make sure my explanation actually captured why old/forgotten API versions and blindly-trusted third-party data create real risk, instead of just restating the definition back.

API Discovery

What I Learned

API Discovery basically means mapping out every endpoint an API has before you start testing it — kind of like scouting a building's full layout before you start checking which locks are weak, so nothing gets missed. I did this two ways: first by reading crAPI's (Completely Ridiculous API, OWASP's intentionally vulnerable practice app) official OpenAPI (Open API Specification, a standard format that documents every endpoint an API has, including its method and expected inputs) spec and importing it into Postman (a tool for organizing and manually sending API requests), and second by just clicking through the live app with Burp Suite's proxy running in the background, passively recording every real request. Comparing the two matters because endpoints sometimes show up in real traffic that were never listed in the official docs — those undocumented endpoints tie directly back to API9 (Improper Inventory Management, from Day 4), since forgotten or untracked endpoints are a common way attackers sneak in.

Tools Used

Postman, Burp Suite Community Edition, MailHog (a fake mail-catcher tool used by crAPI's local lab environment to display "sent" emails, such as vehicle VIN and PIN codes, in a browser instead of a real inbox)

What I Did

Found crAPI's official OpenAPI (Open API Specification) file inside the repo (openapi-spec/crapi-openapi-spec.json) and imported it into Postman, which generated a full collection organized into three service folders matching crAPI's microservice setup: identity, community, and workshop. Separately, logged into crAPI in Firefox with Burp running and manually clicked through the main features — dashboard, vehicle management (including adding a vehicle using a VIN and PIN pulled from MailHog), the shop/purchasing flow, order history, mechanic contact, and community posts — which captured 21 unique live endpoints in Burp's HTTP History. Cross-checked the Burp list against the Postman collection to make sure documented and actual behavior lined up.

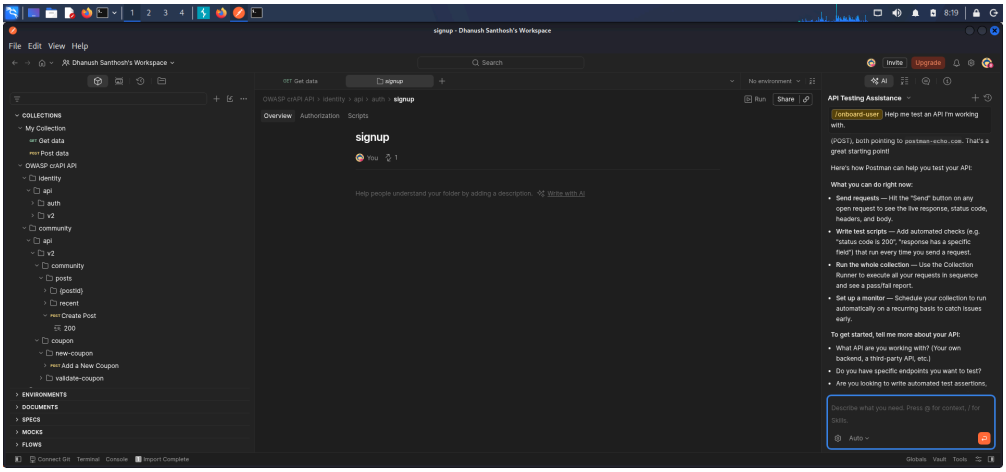


Figure 3: *Postman collection generated from crAPI's official OpenAPI specification, showing endpoints organized into identity, community, and workshop service folders.*

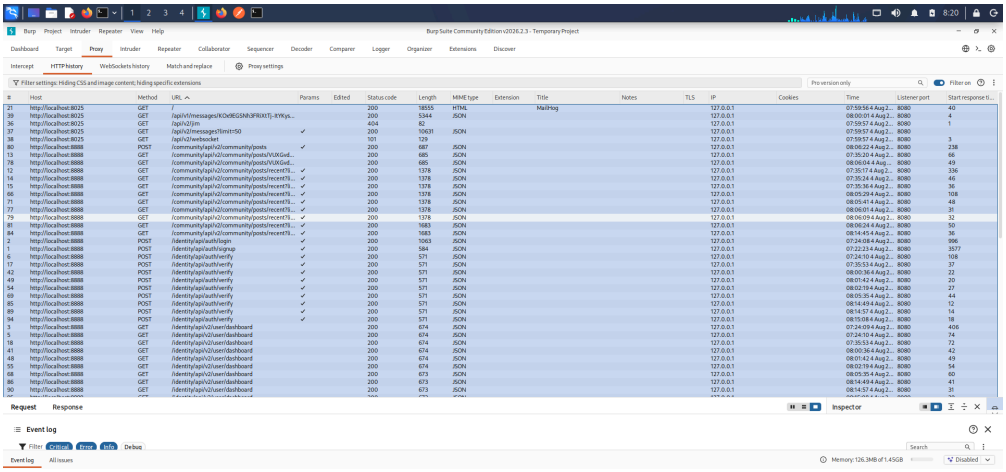


Figure 4: *Burp Suite HTTP History showing crAPI endpoints captured during manual application crawling (part 1).*

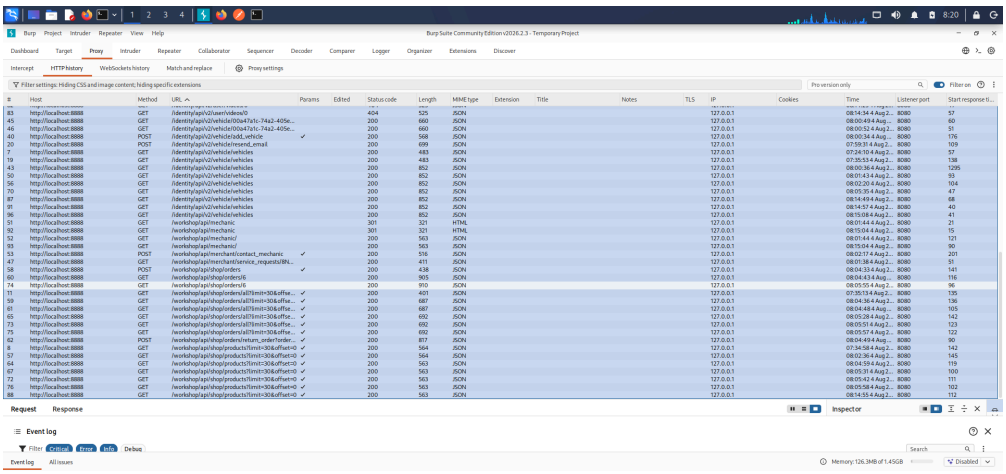


Figure 5: *Burp Suite HTTP History showing crAPI endpoints captured during manual application crawling (part 2), including workshop and shop-related endpoints.*

Findings

Didn't do any actual security testing today, just mapping. One thing worth noting for later: GET /identity/api/v2/user/videos/0 returned a 404, which suggests there's a video-upload feature somewhere in the app that I just didn't find the trigger for. Worth revisiting since video/file upload features are a common source of SSRF (Server-Side Request Forgery, API7 from Day 4) and unrestricted file upload bugs.

Test Cases Added

0 (running total: 0 / 100+) — expected, as structured test case writing begins Day 6.

Challenges

Kept guessing wrong URLs for the OpenAPI docs at first (like /api-docs), which just returned crAPI's blank single-page-app shell instead of a proper 404 — pretty misleading. Fixed it by searching through the crAPI repo's files directly instead of guessing URLs, which is how I actually found the real spec file.

Authentication & JWT Testing

What I Learned

JWT stands for JSON Web Token. It's got three parts: the header, the payload, and the signature. The alg:none attack works because it exploits a server that doesn't actually verify the signature before trusting whatever's in the token — so you can change the header's algorithm to “none,” strip the signature off completely, and a vulnerable server will still accept it like it's a real, legitimate token. That basically bypasses authentication entirely.

Once you know a server skips signature verification, that same flaw opens up a few different attacks. First, just replaying a self-made token bypasses login completely — no real credentials needed. Second, changing the payload's “sub” (subject) field to a different user's email lets you impersonate that person and pull their private data — that's full account takeover, not just bypassing your own login. Third, checking whether logout actually kills a token server-side tests a totally different thing: JWTs are stateless by design, so unless the server keeps its own revocation list, an old token can keep working even after you log out. Fourth, checking whether a token with a past “exp” timestamp still gets accepted tells you whether expiry is enforced server-side at all — though that one often ends up tied to the same broken verification step rather than being a fully separate bug.

Tools Used

Burp Suite Community Edition (HTTP History, Repeater, Decoder), jwt.io, crAPI (two accounts: primary account and a second test account)

What I Did

Grabbed my real JWT from Burp's HTTP History (Authorization header on /identity/api/auth/verify) and decoded it at jwt.io. Header used RS256 (an asymmetric public/private key algorithm), payload had sub, iat, exp, and role claims. Since RS256 uses a private key that can't realistically be brute-forced, skipped the weak-secret attack idea and went for alg:none instead.

Built a forged token by hand in Burp's Decoder — wrote a new header JSON ({"alg":"none","typ":"JWT"}), base64url-encoded it (had to fix up the standard base64 output — swap +/ characters and strip the padding), and stuck it together with the original untouched payload plus an empty signature segment at the end. Sent this against /identity/api/v2/user/dashboard in Repeater and got my full profile data back with a 200 OK — so the server isn't checking the signature at all.

Took it further by making a second crAPI account (testtwo / test2@test.com) through signup + MailHog OTP. Rebuilt the forged token but with the payload's sub changed to the second account's email, everything else the same, double-checked the encoding before sending it. Got that account's private data back — id, email, phone number, available credit — which means full account impersonation, not just bypassing my own auth.

Tested logout by logging out of my own account in the browser, then replaying my original, still-valid, properly-signed token in Repeater — still got a 200 OK with my data back, so crAPI doesn't invalidate tokens server-side on logout. Separately, forged another alg:none token with iat/exp set to January 2020 (way in the past) and sent that too — also got a 200 OK, though that's probably just the same broken verification skipping everything rather than a separate expiry bug.

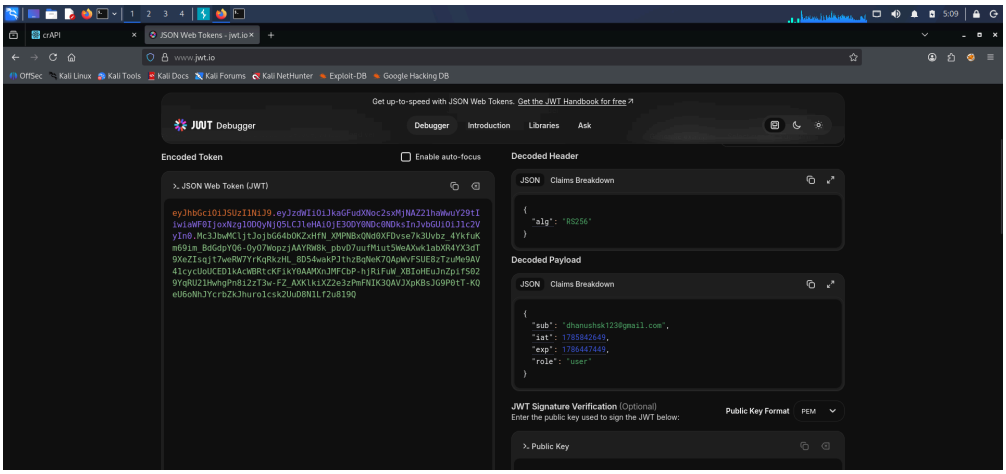


Figure 6: *Decoded JWT at jwt.io showing the RS256 algorithm in the header and the sub, iat, exp, and role claims in the payload.*

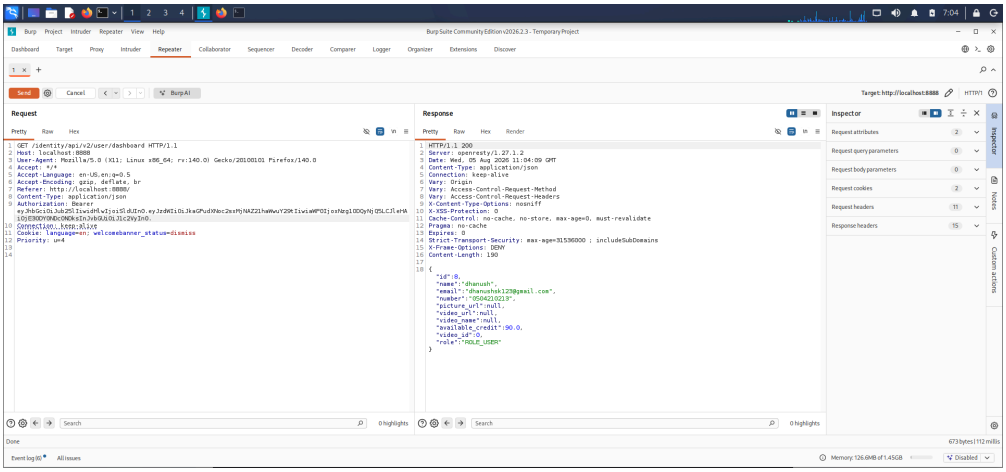


Figure 7: *Burp Repeater response to a forged alg:none token (own payload, signature stripped) — 200 OK returning the authenticated user's full profile data, confirming broken signature verification.*

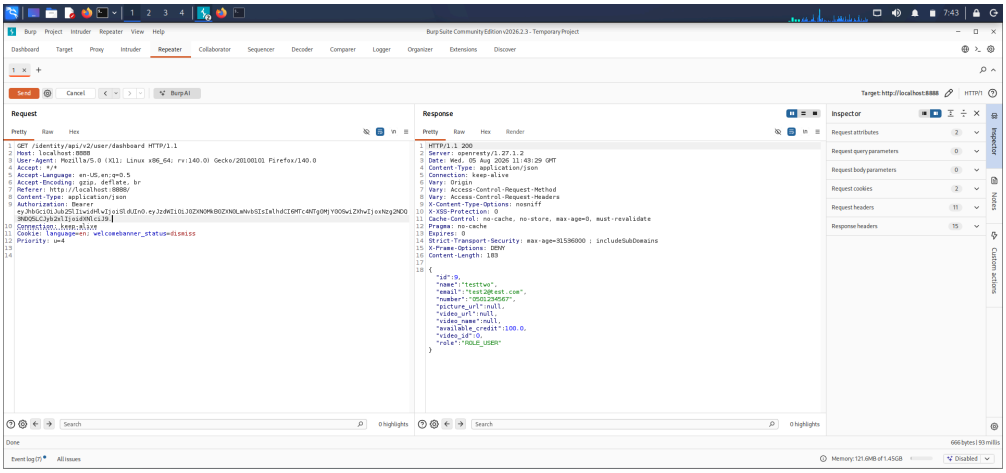


Figure 8: *Burp Repeater response to a forged alg:none token with the payload's sub changed to a second account's email — 200 OK returning that account's private data, confirming full account impersonation.*

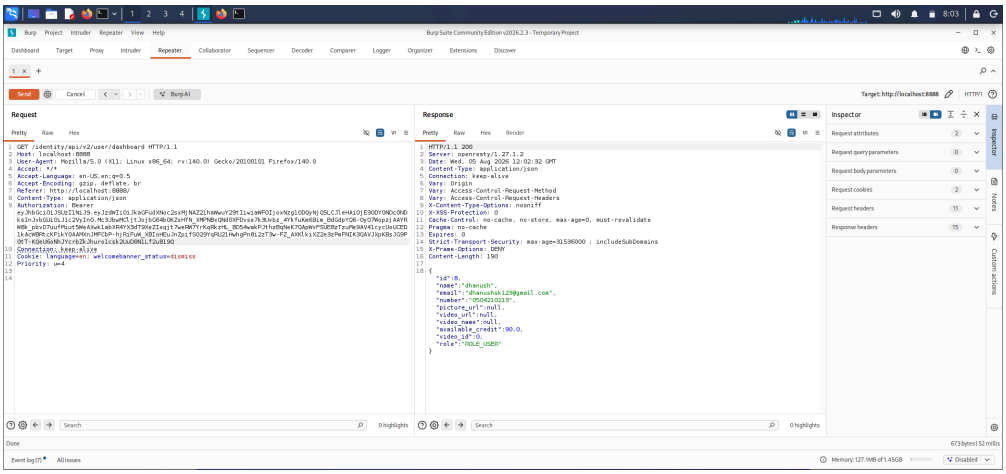


Figure 9: *Burp Repeater response replaying the original, validly-signed JWT after logging out — 200 OK, confirming no server-side session invalidation on logout.*

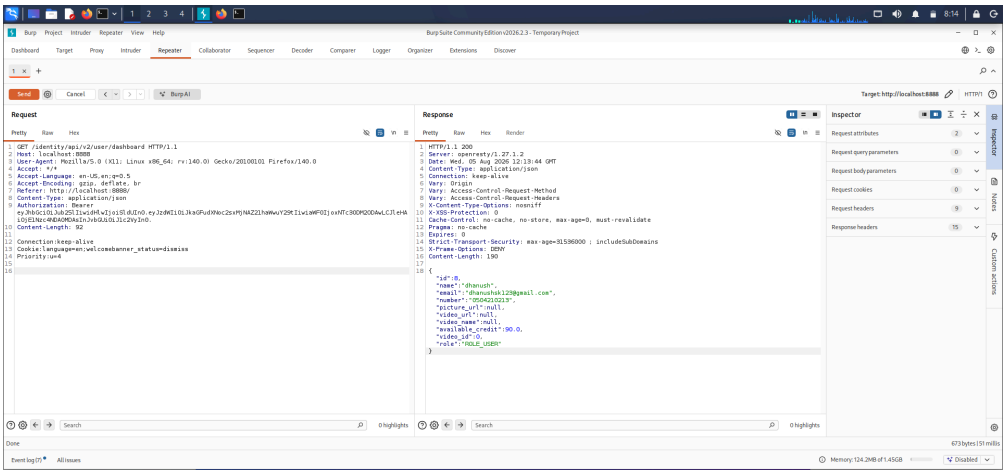


Figure 10: *Burp Repeater response to a forged JWT with an expiry timestamp set in the past (January 2020) — 200 OK, showing the expired token was still accepted.*

Findings

Critical: crAPI doesn't check the JWT signature at all. Just changing the header's alg to “none” and stripping the signature is enough to be treated as fully logged in. Proved this two ways — bypassing my own authentication, and forging a different user's email into the payload to fully impersonate their account and pull their private data (id, email, phone number, available credit) without ever having valid credentials.

Medium: Logging out doesn't invalidate the JWT on the server side. A token issued before I logged out kept working exactly the same afterward, since JWTs are stateless unless the server keeps its own revocation list. That means a stolen/leaked token would stay usable even after the real user logs out.

Note: A token with a past expiry date also got accepted, but since I tested this with an alg:none token, I can't fully separate this from the signature bug above — both are probably being skipped by the same broken verification step rather than being two separate issues. Noting it here rather than calling it out as its own finding for that reason.

Test Cases Added

4 (running total: 4 / 100+)

TC-JWT-01: Modify JWT header alg from RS256 to none, strip signature, replay own payload on /identity/api/v2/user/dashboard — FAIL (200 OK, authentication bypassed with an unsigned token)

TC-JWT-02: Same alg:none technique, forged payload sub to a second user's email (test2@test.com) — FAIL (200 OK, full account impersonation, not just self-auth-bypass)

TC-JWT-03: Replay original validly-signed JWT after logging out — FAIL (200 OK, no server-side session invalidation on logout)

TC-JWT-04: Replay forged alg:none token with exp timestamp set in the past (Jan 2020) — FAIL (200 OK, expired token still accepted; likely tied to the same broken verification step as TC-JWT-01/02)

Challenges

Manually typing out a long JWT from screenshots kept introducing small character errors (mixed up the letter O with the digit 0 at one point), which broke the decoding and gave a misleading “email not registered” error on my first impersonation attempt. Fixed it by copy-pasting the actual token text instead of retyping from screenshots, and by double-checking every forged token decodes back correctly before sending it, instead of only finding out after.

BOLA / IDOR Testing

What I Learned

API1 – BOLA (Broken Object Level Authorization): This is crAPI's signature vuln, and it's basically the same idea as API3 but applied to any object ID in the app, not just user profile fields. If the server only checks "does this ID exist" and never checks "does this ID actually belong to the person asking for it," then I can just swap the ID in a request and pull up someone else's data — a vehicle, a report, a booking, whatever — using nothing but my own valid login. No token forging, no exploit, just changing a value in the URL or request body. Also called IDOR (Insecure Direct Object Reference), and it's why this category matters so much: it doesn't need any clever attack, just patience to try IDs that aren't yours.

Tools Used

Burp Suite Community Edition (Proxy + Repeater), Firefox with FoxyProxy, two separate crAPI accounts (userA and userB)

What I Did

Started by browsing crAPI normally as userA with Burp's HTTP History open, to see which endpoints touch object IDs I could test — landed on two good targets: /identity/api/v2/vehicle/{id}/location (vehicle GPS) and /workshop/api/mechanic/mechanic_report?report_id={id} (mechanic service reports).

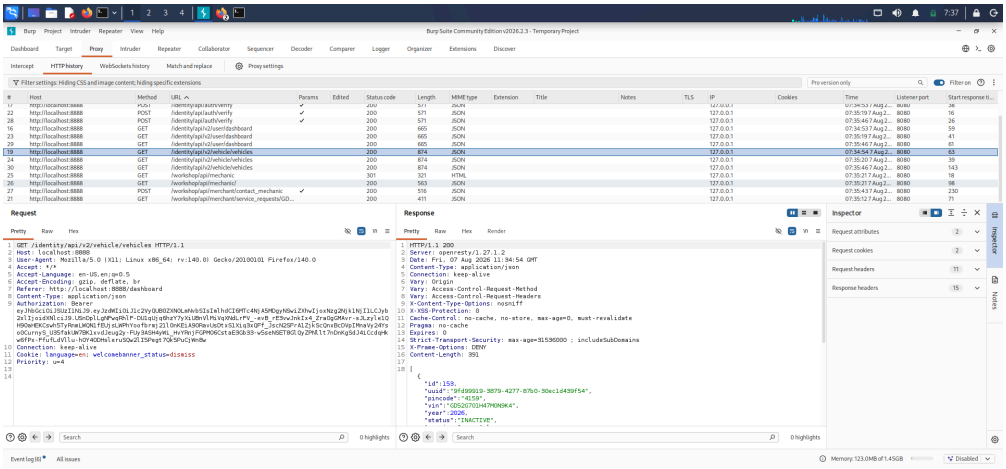


Figure 11: *Burp HTTP History showing the /identity/api/v2/vehicle/vehicles and /workshop/api/mechanic endpoints identified as BOLA/IDOR test targets.*

First test on the vehicle location endpoint: tried a plain sequential number (153) as the ID instead of a real one, just to see how the server reacts to a wrong ID format. Got a 400 Bad Request back — "Failed to convert 'carId' with value: '153'" — which told me crAPI expects a UUID for vehicle IDs, not a small integer. Good to know before wasting time guessing numbers on this particular endpoint.

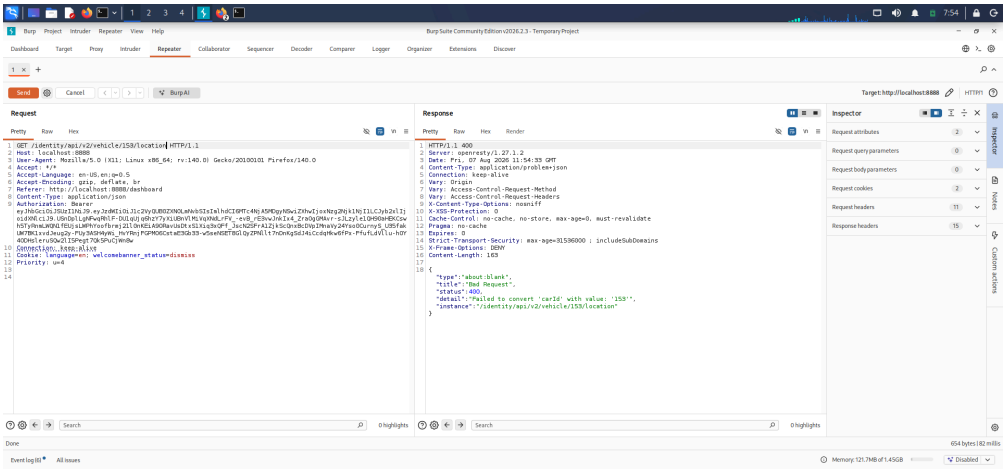


Figure 12: *Vehicle location request using a plain integer ID (153) — 400 Bad Request, confirming crAPI expects a UUID here, not a sequential number.*

Sent the same request again with my own actual vehicle UUID (captured earlier from my own dashboard traffic) — got a clean 200 OK back with my own GPS coordinates, name, and email. That's the expected baseline: I should be able to see my own vehicle's location.

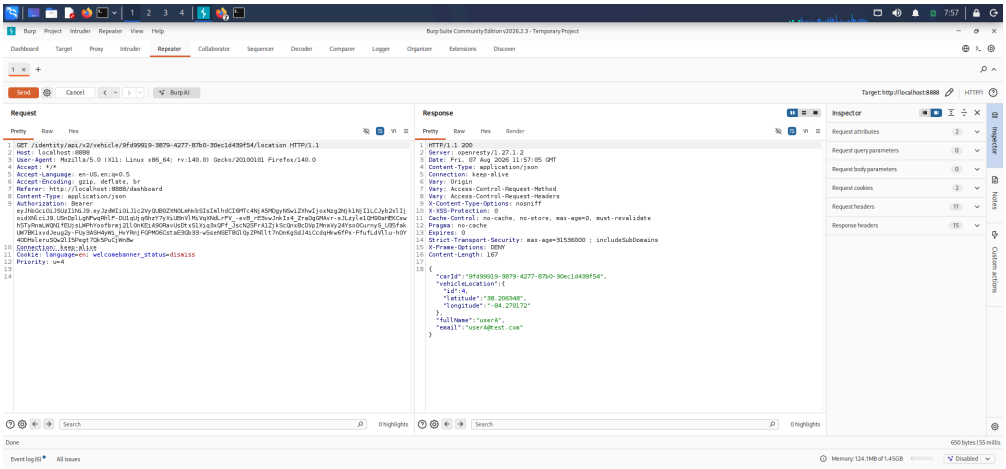


Figure 13: *Vehicle location request using userA's own vehicle UUID — 200 OK, correctly returns userA's own GPS coordinates and identity.*

To get a second UUID to test with, I logged into a separate userB account in a private browsing window (keeping userA's session alive in the normal window, same session-isolation approach from earlier testing) and pulled userB's vehicle UUID off her own dashboard traffic. Then switched back to userA's Repeater tab — still using userA's own Bearer token the whole time — and just replaced the vehicle UUID in the URL with userB's UUID. Got a 200 OK back with userB's GPS location, full name, and email, even though I was authenticated as userA the entire time. That's the BOLA: the server checked that the vehicle ID existed, but never checked that it belonged to me.

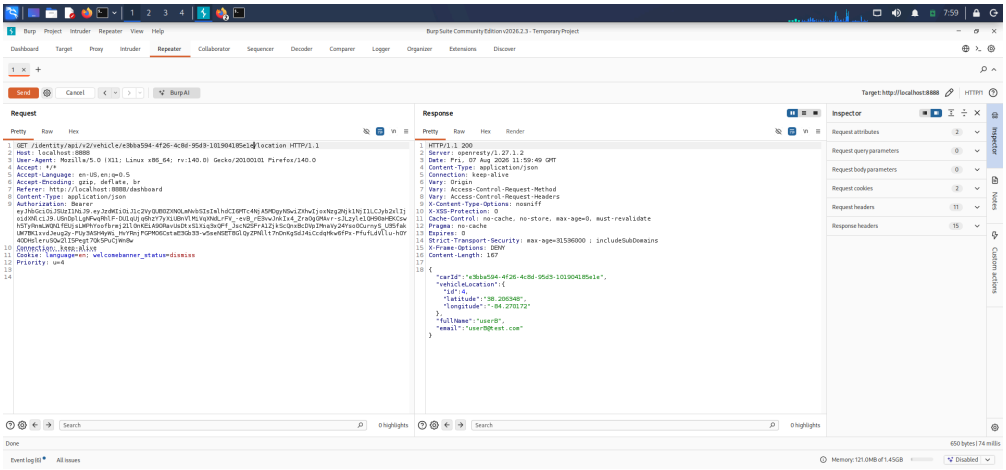


Figure 14: *Vehicle location request using userB's UUID while still authenticated with userA's token — 200 OK, returns userB's GPS coordinates, full name, and email. Confirmed BOLA.*

Moved on to the mechanic report endpoint. Submitted a "contact mechanic" request as userA, which created a new report and returned a report_id (8) in the response, along with a report_link URL I could hit directly to read it back.

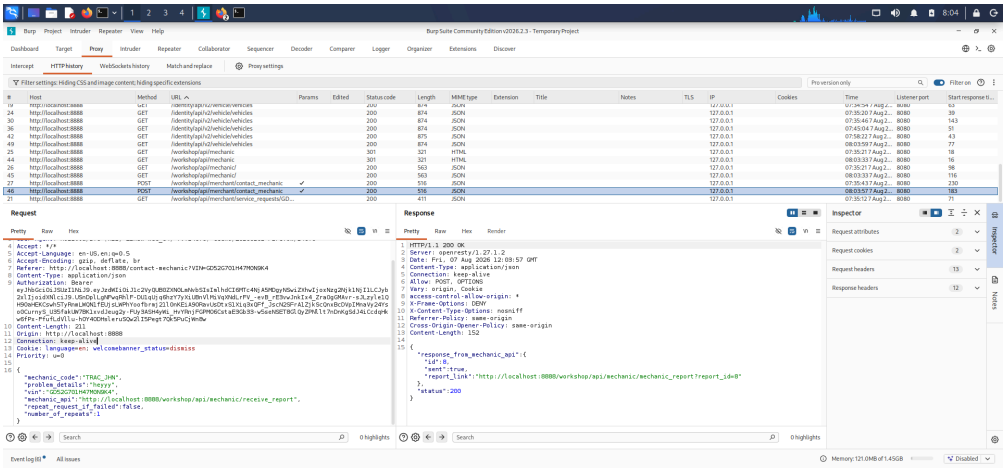


Figure 15: *POST to /workshop/api/merchant/contact_mechanic as userA, creating a new mechanic report and returning report_id: 8.*

Read my own report back with GET /workshop/api/mechanic/mechanic_report?report_id=8 — got a 200 OK with my own vehicle VIN, my own problem description, and my own contact details. Correct baseline, same as the vehicle location test.

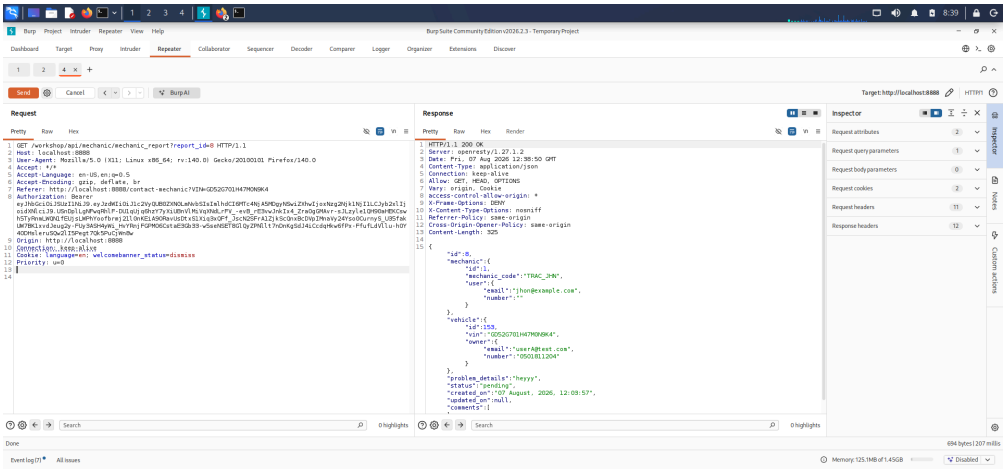


Figure 16: *GET request for report_id=8 (userA's own report) — 200 OK, correctly returns userA's own report details.*

Then tried the same endpoint with report_id=1 instead, still using my own userA token, no re-login. Got a 200 OK back — but this time with a completely different vehicle (VIN 8VAUI03PRUQ686911), a different owner's email, phone number, and their actual problem description text. This is the same BOLA bug as the vehicle location one, but worse in one way: report_id is a small sequential number, not a UUID, so an attacker doesn't even need to find or guess an ID — they can just count 1, 2, 3... and pull every user's mechanic reports off the system one by one.

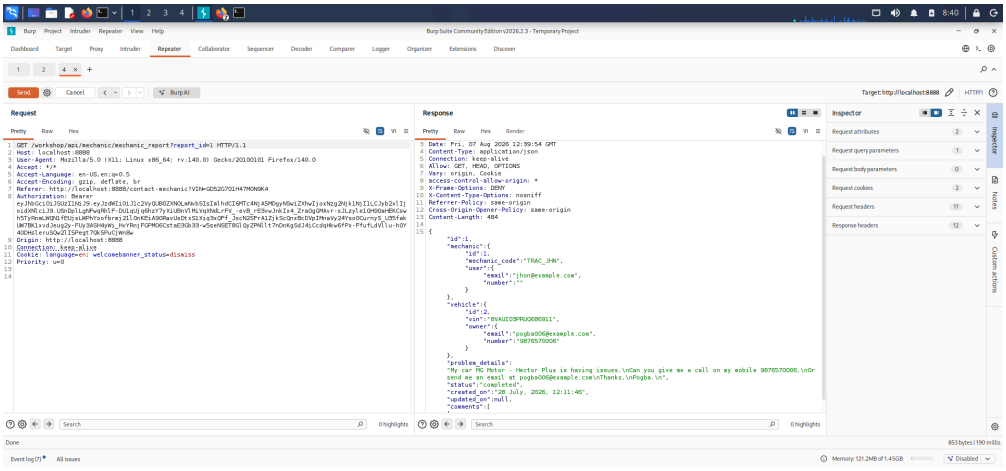


Figure 17: *GET request for report_id=1 while authenticated as userA — 200 OK, returns a completely different user's vehicle VIN, contact details, and problem description. Confirmed IDOR via sequential ID enumeration.*

Side note I want to flag for later: the mechanic report response includes a mechanic_api field with an internal-looking URL (http://localhost:8888/workshop/api/mechanic/receive_report). That's a possible SSRF (Server-Side Request Forgery) angle — a spot where the server itself makes an outbound call using a URL from the request — but I'm deliberately leaving it alone for now since SSRF testing is scheduled for Day 11 and I don't want to test it half-prepared.

Findings

Finding 1 — BOLA on vehicle location endpoint (High): Authenticated as userA, was able to retrieve userB's live vehicle GPS location, full name, and email by simply substituting userB's vehicle UUID into the request URL. The server validates that the ID exists but never checks it belongs to the requesting user.

Finding 2 — IDOR on mechanic report endpoint via sequential ID (High): Authenticated as userA, was able to retrieve another user's mechanic service report (vehicle VIN, phone number, and problem description) by changing report_id to a low sequential number (1) instead of my own report's ID (8). Rated High rather than Critical since it exposes PII and vehicle data but not credentials or account takeover — but the sequential ID makes it trivially enumerable, which is a serious real-world risk on its own.

Test Cases Added

2 (running total: 6 / 100+)

TC-BOLA-01: Authenticated as userA, requested /identity/api/v2/vehicle/{userB's UUID}/location — FAIL (200 OK, returned userB's GPS location, name, and email to userA's session)

TC-BOLA-02: Authenticated as userA, requested /workshop/api/mechanic/mechanic_report?report_id=1 (not userA's own report_id of 8) — FAIL (200 OK, returned a different user's VIN, phone number, and problem description)

Challenges

First attempt at the vehicle location endpoint used a plain number (153) as the ID and got a 400 error, which had me worried the endpoint wasn't vulnerable at all — turned out I just had the wrong ID format, and switching to the actual UUID from my own dashboard traffic fixed it. Good reminder to check the expected ID format before concluding an endpoint isn't testable.

Broken Function Level Authorization (BFLA) + Excessive Data Exposure / Mass Assignment

Note: per an updated deadline from my supervisor [redacted], the internship now needs to be fully wrapped up and submitted by Friday, August 14, 2026 instead of the original August 21 date. From here on, days combine two or more OWASP categories to fit the compressed timeline, rather than one category per day.

What I Learned

API5 – Broken Function Level Authorization (BFLA): This is different from BOLA (Day 7). BOLA is "can I see someone else's data" — BFLA is "can I call a function I'm not supposed to have access to at all," usually something meant for admins or another privileged role, using nothing but my own normal, low-privilege token. The server checks that I'm logged in, but never checks whether someone with my role should be allowed to call that specific function in the first place.

API3 – Mass Assignment (revisited): The API accepts more fields from the client than it should, letting a user sneak in a field they're not supposed to control — like adding "role":"admin" to a signup or profile update request, hoping the server blindly saves whatever it's given instead of only the fields it actually meant to expose.

Tools Used

Burp Suite Community Edition (Proxy + Repeater), Firefox with FoxyProxy, ffmpeg (to generate a test video file), crAPI accounts (userA, and new accounts userC/userD created during testing)

What I Did — BFLA

Started by trying to find real admin-only endpoints instead of guessing blindly. Grepped crAPI's OpenAPI spec file for the word "admin" and found one clearly admin-tagged operation: DELETE /identity/api/v2/admin/videos/{video_id} — "Delete Profile Video Admin," tagged "Identity / Admin," meant for ROLE_ADMIN only (the spec also showed the role enum: ROLE_UNDEFINED, ROLE_USER, ROLE_MECHANIC, ROLE_ADMIN).

First sent the request with a random non-existent video ID (999999) using my normal userA token, just to see how the server reacts before committing to anything real.

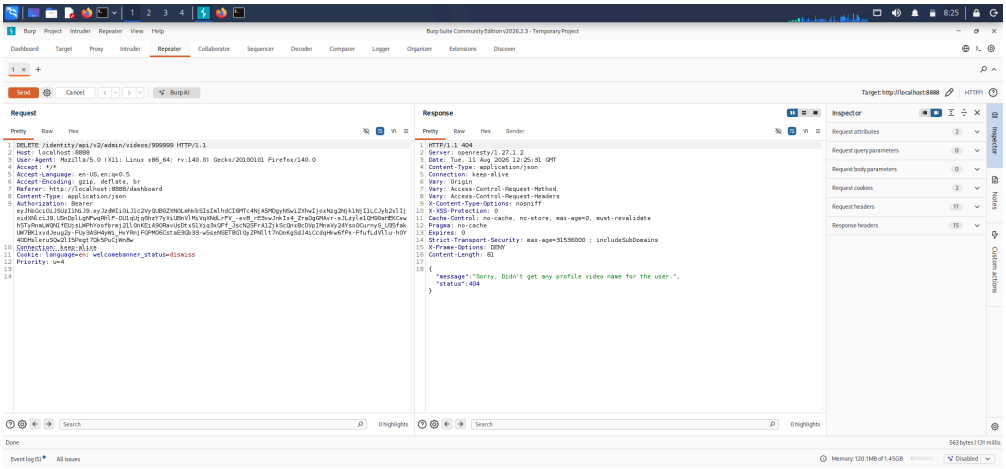


Figure 18: *DELETE request to the admin-only video-delete endpoint with a non-existent video ID (999999), using userA's normal token — 404 "Didn't get any profile video name for the user," not 403. This means the server never checked userA's role at all; it went straight to looking for the video and only failed because that ID doesn't exist.*

A 404 instead of a 403 is exactly the BFLA signature — the server skipped the role check entirely and went straight to "does this ID exist," so I needed to prove it with a real video ID to be sure. Generated a small test video locally with ffmpeg (no need to download anything from the internet) and uploaded it through crAPI's profile video feature as userA, which returned video_id: 52.

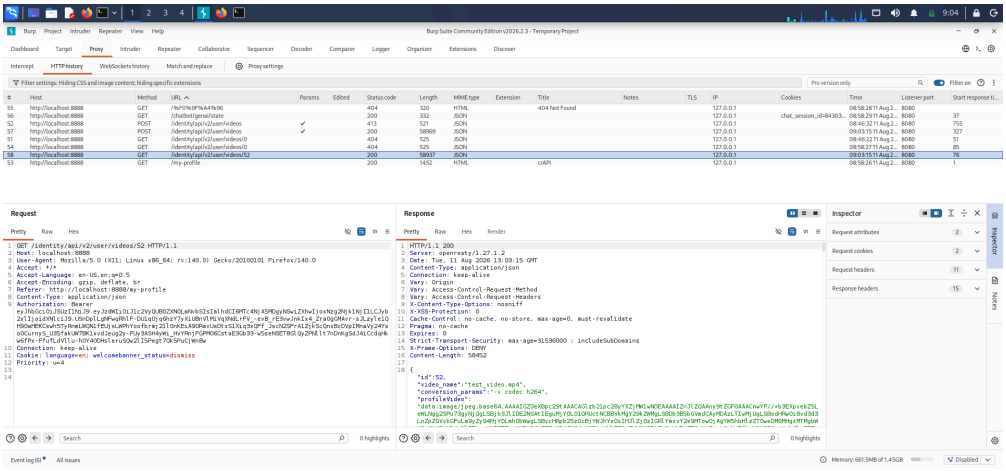


Figure 19: *GET request confirming video_id 52 exists and belongs to userA, returned via the normal user videos endpoint.*

Sent the same DELETE request again, this time with the real ID (52), still using userA's ordinary token — no elevated permissions of any kind.

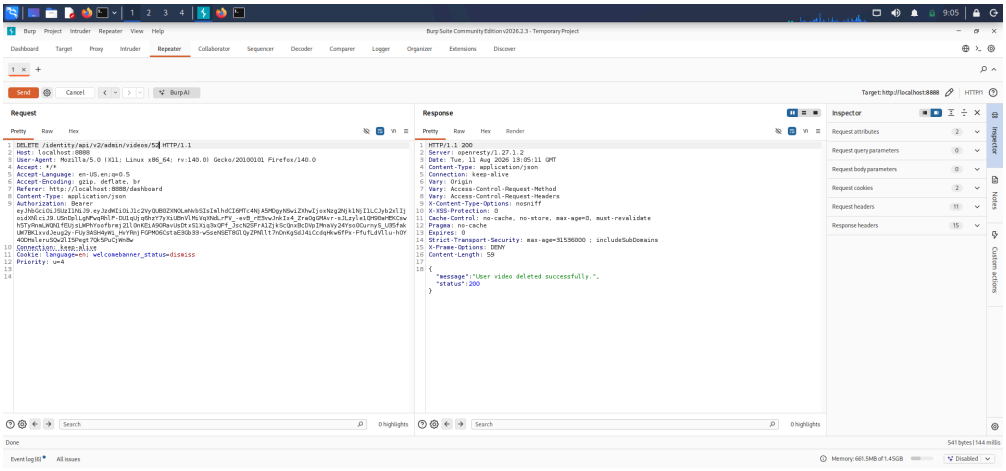


Figure 20: *DELETE /identity/api/v2/admin/videos/52 using userA's token — 200 OK, "User video deleted successfully." A regular customer account successfully executed a genuine admin-only destructive action.*

To rule out a fake success message, sent a fresh GET for the same video ID right after.

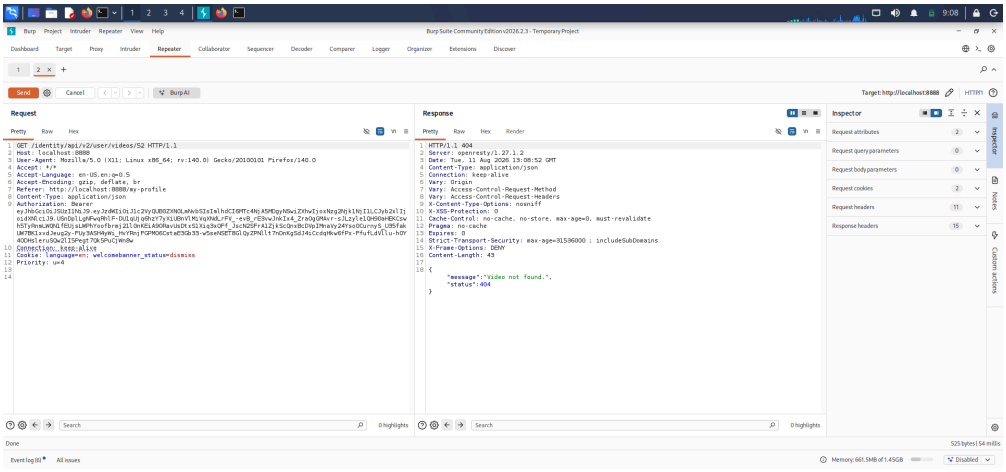


Figure 21: *Follow-up GET for video_id 52 — 404 "Video not found," confirming the deletion actually happened and wasn't just a false success response.*

Checked the OpenAPI spec for any other endpoints tagged "Identity / Admin" to see if there were more BFLA targets worth testing, but the video-delete endpoint is the only one tagged that way in the whole spec, so this is the one clean, fully-proven finding for this category.

What I Did — Mass Assignment

Picked the phone number change feature on my profile page as the first test target, since it's the only editable field besides email. Captured the normal request first to see the real field names it sends: old_number and new_number, nothing else.

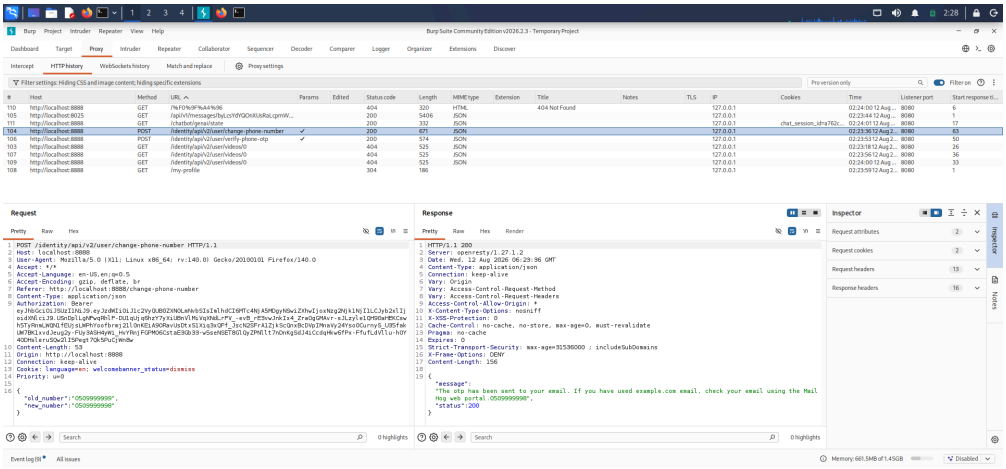


Figure 22: *Captured POST /identity/api/v2/user/change-phone-number request showing the baseline field names (old_number, new_number) before any tampering.*

Resent the request in Repeater with an extra field added — "role":"admin" — alongside the normal old_number/new_number values, to see if the server would silently accept and apply a field it was never supposed to receive.

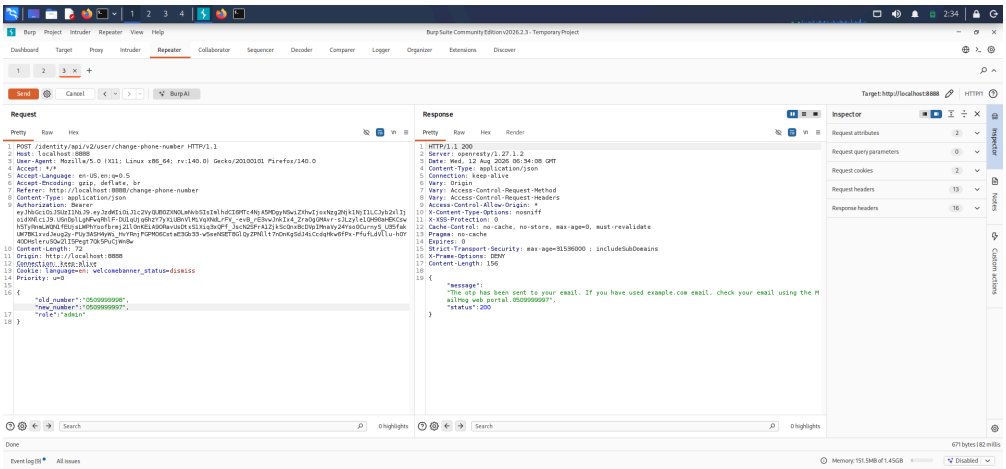


Figure 23: *Same phone-change request with an added "role":"admin" field — server accepted it with 200 OK (OTP sent), no rejection for the unexpected field.*

A 200 OK alone doesn't prove anything though — the extra field could just be silently ignored rather than actually applied. Checked userA's dashboard endpoint right after to see if the role had actually changed.

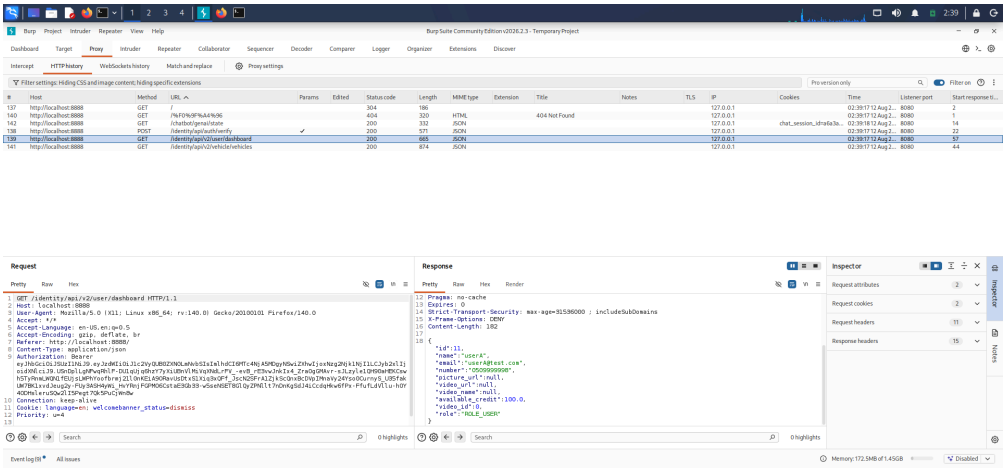


Figure 24: *GET /identity/api/v2/user/dashboard for userA after the tampered request — role still shows "ROLE_USER", confirming the extra field was ignored, not applied. PASS (not vulnerable) on this endpoint.*

Mass Assignment is often more exploitable during account creation than on an update to an existing account, so tried the same idea on signup instead of assuming one negative result covered the whole app. Captured a normal signup request first (name, email, number, password), then resent it in Repeater for a brand-new account (userD) with an extra "role":"admin" field added to the body.

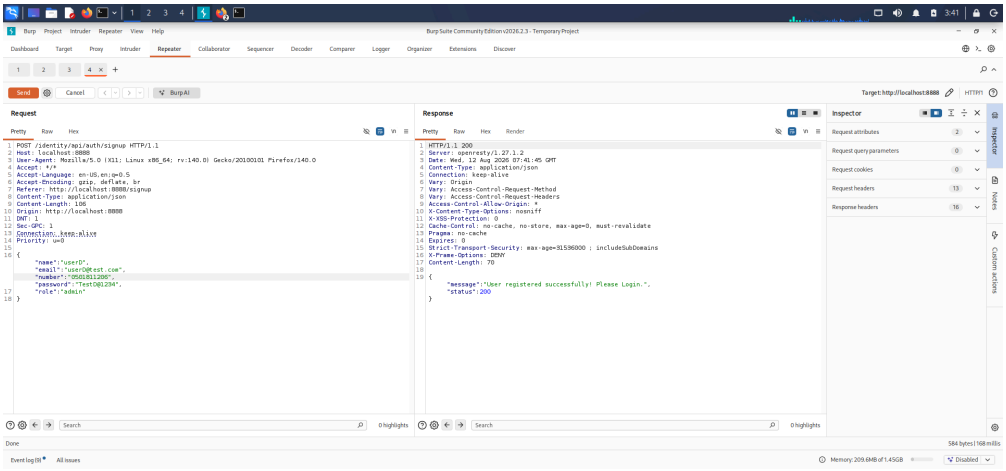


Figure 25: *POST /identity/api/auth/signup for a new account (userD) with an added "role":"admin" field in the request body — 200 OK, "User registered successfully!"*

Logged in as the newly created userD and checked its dashboard role field the same way.

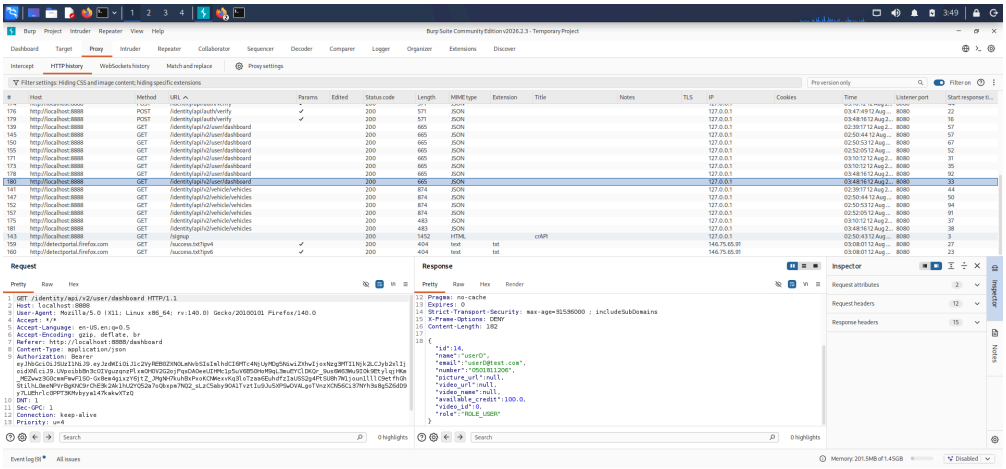


Figure 26: *GET /identity/api/v2/user/dashboard for the newly created userD account — role shows "ROLE_USER", confirming the tampered role field was ignored on signup as well. PASS (not vulnerable).*

Findings

Finding 1 — BFLA on admin video deletion endpoint (Critical): A regular ROLE_USER account (userA) was able to successfully call DELETE /identity/api/v2/admin/videos/{video_id} — an endpoint explicitly documented as ROLE_ADMIN-only — and permanently delete a real profile video, with no role check enforced server-side at any point. Proven end-to-end: uploaded a real video, confirmed it existed, deleted it via the admin endpoint using userA's normal token, and confirmed via a follow-up GET that it was genuinely gone. Rated Critical rather than High since this allows actual data destruction (not just exposure) via an action that should require elevated privileges.

Note — Excessive Data Exposure on video endpoint (Low/Informational): The user videos response embeds the full profile video as a base64-encoded blob directly inside a JSON response (profileVideo field), rather than serving it as a separate file/stream. Not a serious vulnerability on its own, but unusual API design that bloats every response containing this field and isn't a real security control — noting it for completeness rather than as a standalone finding.

Mass Assignment — tested, not vulnerable (Pass): Tried injecting an unexpected "role":"admin" field into both the profile phone-number-change request and a new-account signup request. In both cases, the server correctly ignored the extra field and the account's role remained ROLE_USER. This endpoint and the signup flow both properly validate/whitelist input fields rather than blindly trusting the request body.

Test Cases Added

3 (running total: 9 / 100+)

TC-BFLA-01: Authenticated as userA (ROLE_USER), sent DELETE /identity/api/v2/admin/videos/{video_id} for a real video ID — FAIL (200 OK, video deleted successfully; confirmed with follow-up GET returning 404)

TC-MASS-01: Sent POST /identity/api/v2/user/change-phone-number with an added "role":"admin" field — PASS (field ignored, role stayed ROLE_USER on dashboard check)

TC-MASS-02: Sent POST /identity/api/auth/signup for a new account with an added "role":"admin" field — PASS (field ignored, new account's role stayed ROLE_USER on dashboard check)

Challenges

Didn't have a video file handy to test the upload feature and didn't want to download a random one from the internet into the lab environment, so generated a small test clip locally with ffmpeg instead — quick and self-contained.

Hit a couple of small mistakes along the way that ate some time: edited a live Repeater request in place at one point and it returned an unrelated "Invalid Token" error, which turned out to be a corrupted request rather than an actual auth issue — fixed by re-sending a clean copy from HTTP History instead of editing in place. Also tripped the phone-number-change endpoint's own validation twice by forgetting to update old_number to my actual current number after a previous successful change, and by leaving out a comma when manually adding the extra role field to the JSON body — small syntax slip, not a security-relevant result, just needed to fix the JSON and resend.

Rate Limiting + SSRF + Security Misconfiguration + SQL Injection

What I Learned

API4 – Unrestricted Resource Consumption (Rate Limiting): Checking whether the server puts any limit on how many times a request can be sent — failed logins, OTP (One-Time Password) requests, etc. If there's no limit, an attacker can brute-force logins or spam OTP generation with nothing stopping them.

API7 – SSRF (Server-Side Request Forgery): This is when a feature makes the server itself fetch a URL on your behalf, and you can trick it into fetching an internal address instead of the one it was meant to call. If the server hands back the response from that call, you can potentially read data from internal systems that should never be reachable from outside.

API8 – Security Misconfiguration: Basic security setup mistakes — verbose error messages, missing security headers (like CSP, Content Security Policy — a header telling the browser which sources are allowed to run scripts/styles on the page), or overly permissive CORS (Cross-Origin Resource Sharing — a browser rule about which websites are allowed to call this API).

SQL Injection (SQLi): Sneaking SQL syntax into a field (like login email) so the database runs an attacker's extra command instead of treating the input as plain data. Classic test: entering something like ' OR '1'='1 into a login field to try to bypass authentication by making the underlying query always evaluate true.

Tools Used

Burp Suite Community Edition (Proxy + Repeater), Firefox with FoxyProxy, MailHog, crAPI account (userA)

What I Did — Rate Limiting

Captured a failed login attempt for userA@test.com (wrong password) on POST /identity/api/auth/login, sent it to Repeater, and fired it 15 times in a row without changing anything.

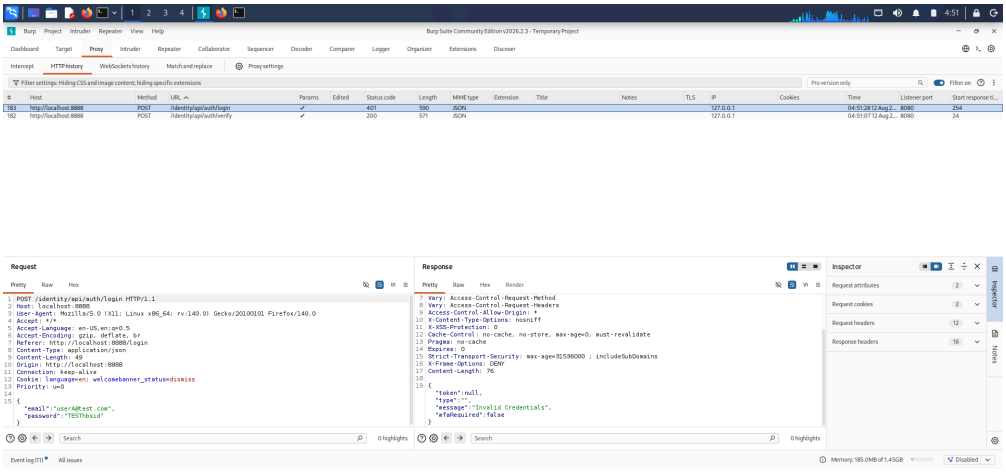


Figure 27: *Burp Proxy HTTP History and Repeater showing repeated POST /identity/api/auth/login attempts with a wrong password — every attempt returned an identical 401 "Invalid Credentials" response, with no delay or lockout.*

Every single attempt came back with the exact same 401 response, no delay, no CAPTCHA, no lockout message. Noticed in the response headers that Access-Control-Allow-Origin is set to * (wide open CORS) — flagged for the Security Misconfiguration section below.

Repeated the same idea against the OTP-generation endpoint. Triggered "Forgot Password" for userA@test.com and sent the request 20 times in a row.

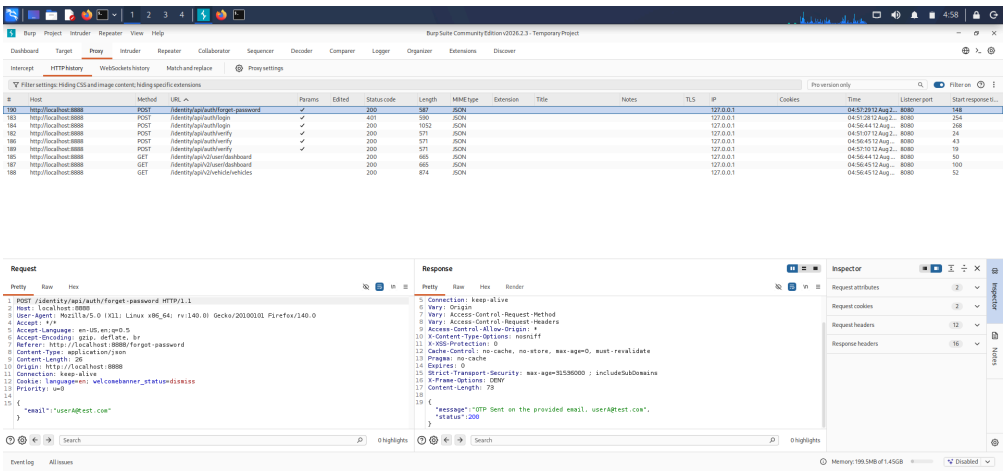


Figure 28: *POST /identity/api/auth/forget-password sent repeatedly for userA@test.com — every attempt returned 200 OK with "OTP Sent on the provided email", confirming no rate limit on OTP generation.*

Same result every time — 200 OK, "OTP Sent on the provided email". No limit on how many OTP emails can be triggered for the same address.

Wanted to go a step further and test rate limiting on OTP verification itself (guessing the code), since that's the more serious version of this bug — if the code itself can be brute-forced, an attacker doesn't need to intercept the real email at all. Went through the actual "Forgot Password" UI flow with a real OTP from MailHog to find the verification endpoint.

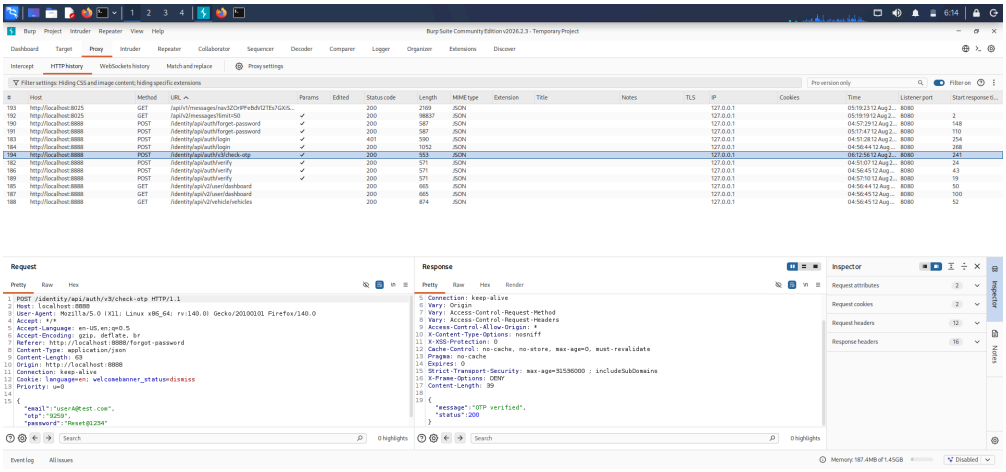


Figure 29: *POST /identity/api/auth/v3/check-otp with the real 4-digit OTP (9259) for userA@test.com — 200 OK, "OTP verified", revealing the endpoint and request format (email, otp, password fields).*

Noted that crAPI's OTPs are only 4 digits (10,000 possible combinations) — much easier to brute-force than a typical 6-digit code if this endpoint has no rate limiting either. Sent the same request to Repeater with the OTP changed to a wrong value (0000) to test that.

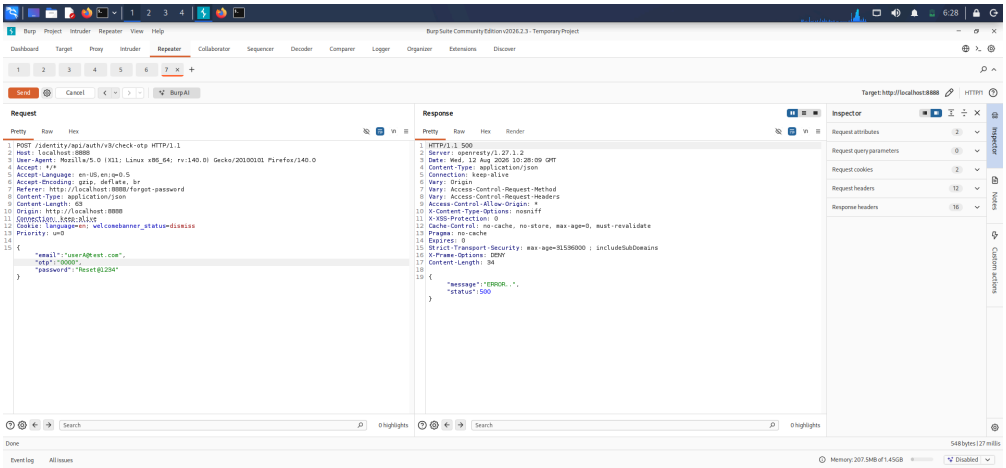


Figure 30: *POST /identity/api/auth/v3/check-otp with a wrong OTP (0000) — instead of a clean rejection, the server returned a 500 Internal Server Error with a raw generic message, both against an already-used session and a freshly generated one.*

Instead of a clean "invalid OTP" rejection, the server crashed with a raw 500 error every time a wrong OTP was submitted — tried this against both an already-completed reset session and a freshly generated one, same result both times. This is a real bug (unhandled input case), but it also got in the way of properly testing brute-force rate limiting on this specific endpoint, since the crash happens before any lockout logic would even apply.

What I Did — SSRF

Went back to the mechanic_api field flagged during Day 7's BOLA testing. Re-captured the POST /workshop/api/merchant/contact_mechanic request and checked the full request body this time (not just the response) to confirm whether the URL is client-supplied.

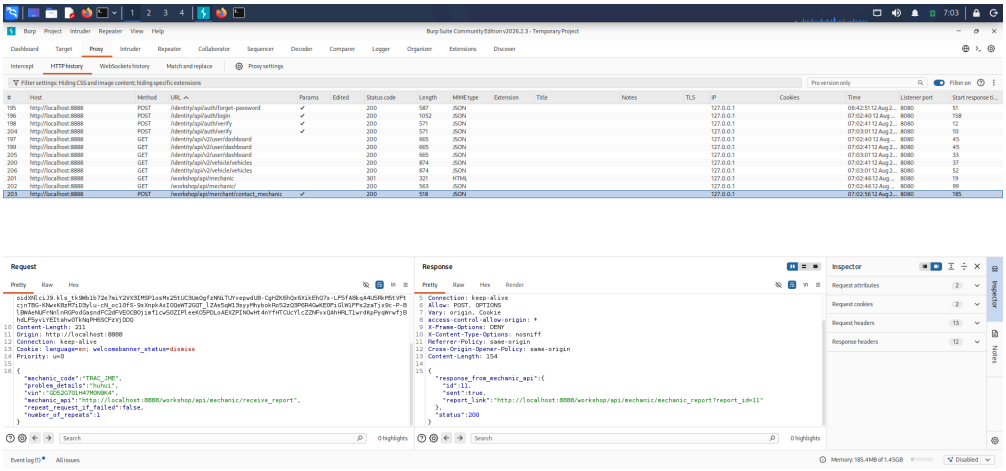


Figure 31: *Full request body for POST /workshop/api/merchant/contact_mechanic, showing the mechanic_api field ("http://localhost:8888/workshop/api/mechanic/receive_report") is sent directly by the client — confirming this is attacker-controlled input, not server-generated.*

Confirmed mechanic_api is a field the client sends, and the response includes a response_from_mechanic_api field that echoes back whatever the server received when it called that URL — a textbook SSRF setup. Sent the request to Repeater and tried substituting different URLs in the mechanic_api field:

First tried http://127.0.0.1:8888/workshop/api/mechanic/receive_report (same target as the original, just IP instead of hostname) — got "Could not connect to mechanic api", inconsistent with the working localhost version. Then tried http://169.254.169.254/latest/meta-data/ (the cloud metadata address, relevant on real cloud infrastructure) — also failed to connect, expected since there's nothing listening there in this local lab setup.

Then pointed mechanic_api at a completely different internal endpoint to see if the server would actually reach and return it: http://localhost:8888/identity/api/v2/user/dashboard.

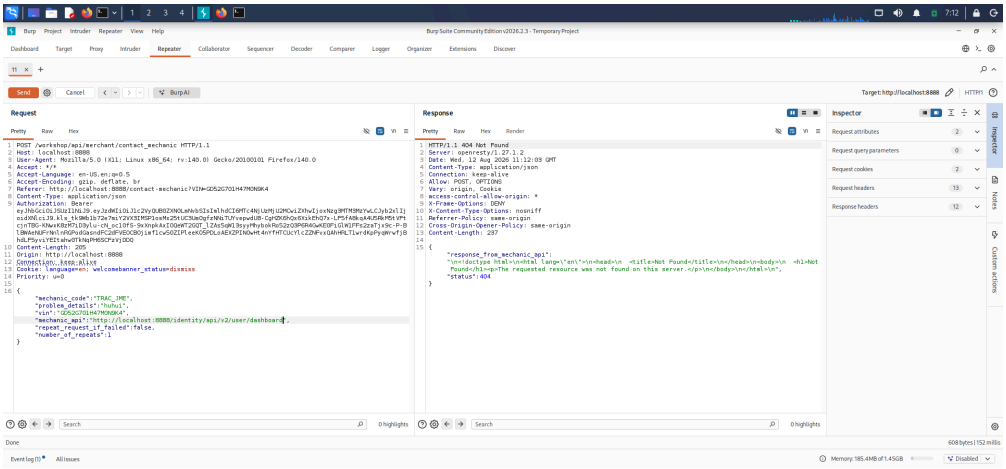


Figure 32: *mechanic_api changed to an unrelated internal endpoint (/identity/api/v2/user/dashboard) — server made the outbound call and reflected the full raw response (a 404 HTML page) back inside response_from_mechanic_api, confirming the server will fetch and return content from any URL supplied in this field.*

Got a 404 back, but the important part is that the server actually made the call and reflected the real response content back to me — confirming SSRF: the server isn't restricted to only calling its own mechanic service, it will fetch whatever URL is supplied in this field and hand back the result.

Tried pushing further to reach an authenticated internal endpoint (/identity/api/v2/vehicle/vehicles) to see if the server-side call carried any authentication that could leak real data.

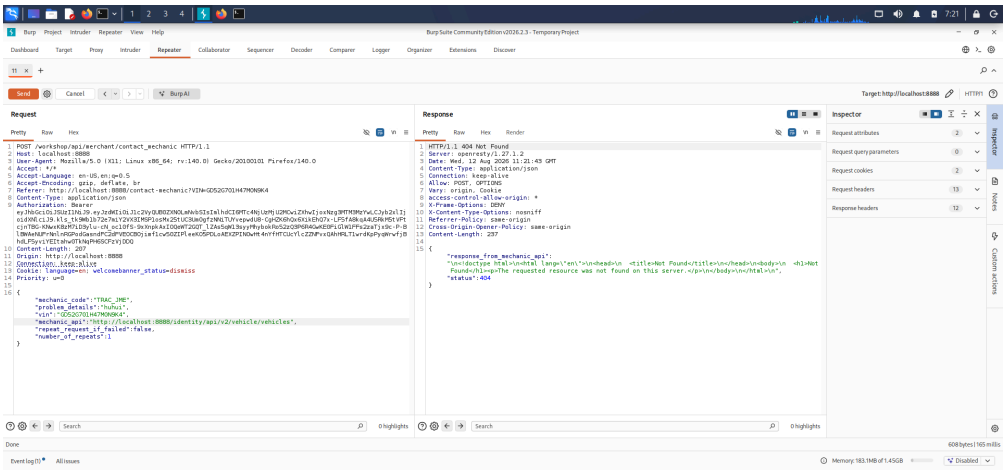


Figure 33: *mechanic_api changed to an authenticated internal endpoint (/identity/api/v2/vehicle/vehicles) — same generic 404 HTML page reflected back, consistent with the previous test rather than a new result.*

Got the same generic 404 HTML page again. This looks like a routing limitation specific to this lab's Docker networking (the workshop service can't reach /identity/* paths via localhost:8888 the way a browser can) rather than a security control blocking the request — so I didn't pursue further internal

endpoint enumeration. The core SSRF capability is already proven by the successful reflected response above.

What I Did — Security Misconfiguration

Sent a deliberately malformed JSON body (missing proper structure) to POST /workshop/api/merchant/contact_mechanic to check how the server handles bad input.

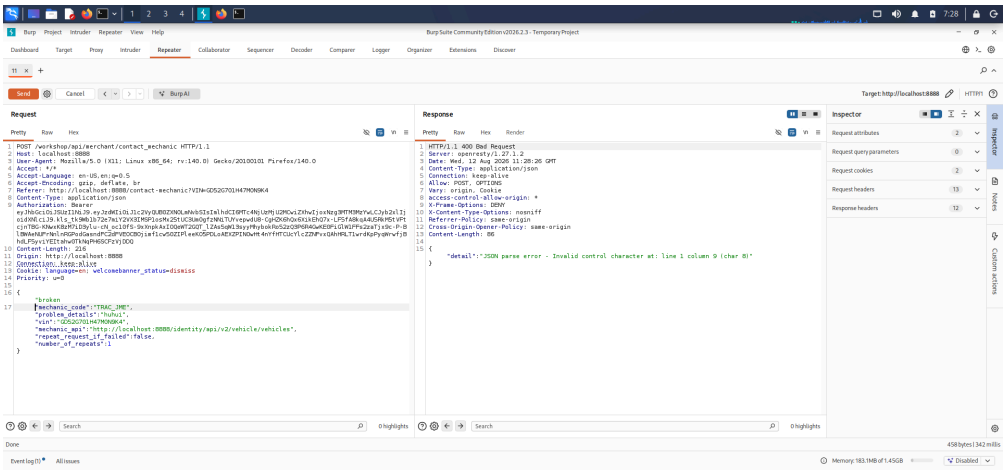


Figure 34: *Malformed JSON body sent to /workshop/api/merchant/contact_mechanic — server returned a clean 400 Bad Request with a short generic parse-error message, no stack trace or internal file paths exposed.*

Got a clean 400 Bad Request with a short, sensible "JSON parse error" message — no stack trace, no internal file paths, no framework details leaked. This one's a pass, worth documenting since it shows this particular error path is handled properly, unlike the check-otp 500 error above.

Checked response headers across multiple endpoints tested today for Content-Security-Policy (CSP) — a header that restricts which sources a browser is allowed to load scripts/styles from. Did not find it present on any response, on top of the wide-open CORS (Access-Control-Allow-Origin: *) already noted during the rate-limiting tests.

What I Did — SQL Injection

Tried classic SQL injection payloads in the email field on POST /identity/api/auth/login. First attempt used the payload without proper JSON string quoting, which just broke JSON parsing entirely ("Failed to read request") rather than testing anything — fixed the request so the payload was a valid JSON string value before re-testing.

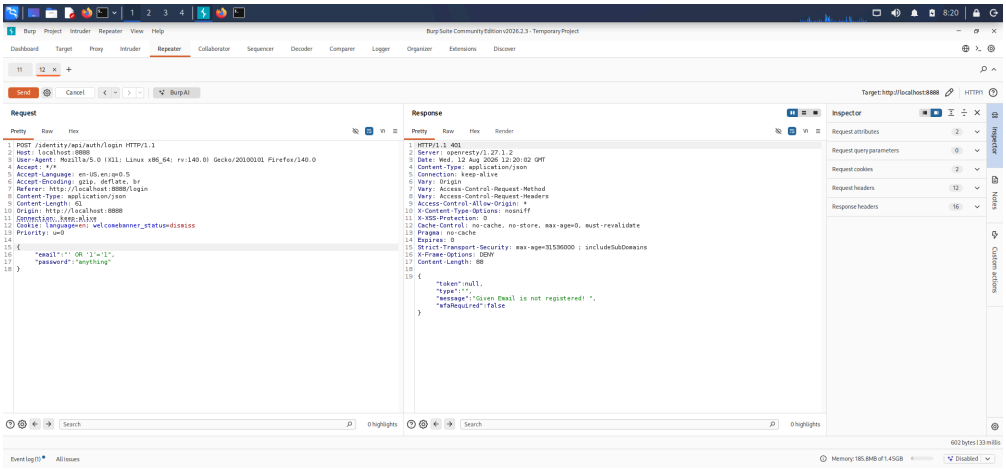


Figure 35: *POST /identity/api/auth/login with email set to the JSON string "' OR '1'='1" — server returned "Given Email is not registered!", treating the payload as a literal invalid email rather than executing it as SQL.*

Server treated the whole payload as a literal (invalid) email string and returned "Given Email is not registered!" — no authentication bypass, no database error. Suggests parameterized queries (a safe way of writing database code where user input is always treated as data, never as part of the command) are in use here. Tried a second classic payload, the comment-based bypass, to confirm this wasn't a one-off.

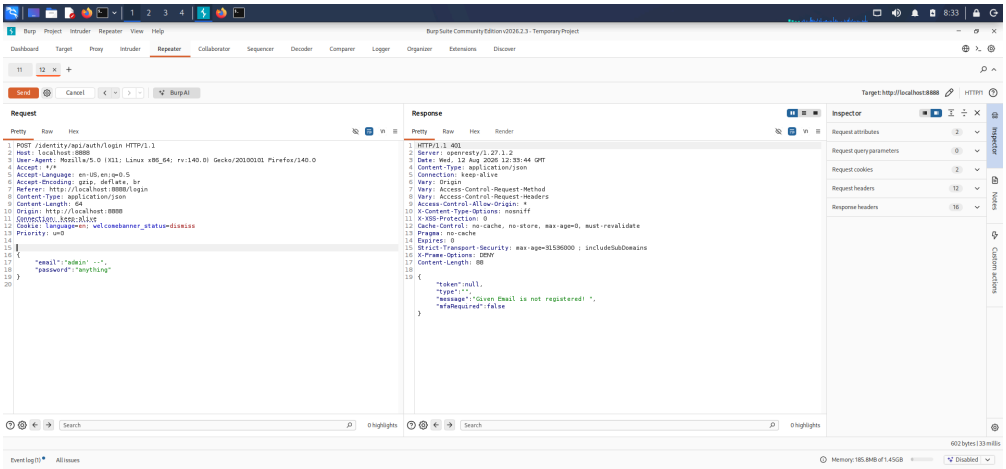


Figure 36: *POST /identity/api/auth/login with email set to "admin' --" (SQL comment-injection payload) — same safe result: "Given Email is not registered!", no query manipulation.*

Same safe result. Two consistent passes on the login endpoint's email field is a reasonable place to stop testing SQLi there — login fields tend to be the most hardened part of an app anyway. Ran out of time to extend SQLi/NoSQLi testing to the community search/filter endpoints, which are usually a weaker target — carried over to tomorrow.

Findings

Finding 1 — No rate limiting on login or OTP generation (High): The login endpoint (/identity/api/auth/login) and the OTP-generation endpoint (/identity/api/auth/forget-password) both

accept unlimited repeated requests with no delay, CAPTCHA, or lockout. This allows unrestricted brute-force attempts against passwords and enables email/OTP-spam abuse against any registered account.

Finding 2 — Confirmed SSRF on mechanic report feature (High): The mechanic_api field in POST /workshop/api/merchant/contact_mechanic is client-controlled and used by the server to make its own outbound HTTP request. Proven by substituting an unrelated internal endpoint (/identity/api/v2/user/dashboard) into this field and observing the server's real response reflected back in response_from_mechanic_api. Could not confirm deeper impact (e.g. reaching an authenticated internal service) within this lab's networking setup, but the core SSRF capability — server fetching and returning content from an attacker-supplied URL — is clearly demonstrated.

Finding 3 — Security Misconfiguration: open CORS and missing CSP (Medium):
Access-Control-Allow-Origin is set to * across tested endpoints, meaning any website can make cross-origin requests to this API from a victim's browser. Content-Security-Policy is also absent from all responses checked, removing a browser-level defense layer against script injection.

Bug — check-otp endpoint crashes on invalid input (Low, reliability/misconfiguration): POST /identity/api/auth/v3/check-otp returns a raw 500 Internal Server Error instead of a clean 400 when given a wrong OTP, both for an already-used and a freshly generated session. Not itself a data-exposure bug, but improper error handling, and it prevented reliable brute-force/rate-limit testing on this specific endpoint since the crash occurs before any lockout logic would apply.

SQL Injection — tested, not vulnerable (Pass): Classic SQLi payloads (' OR '1'='1 and admin' --) on the login email field were both safely treated as literal strings with no query manipulation or authentication bypass, suggesting parameterized queries are in use on this endpoint.

Test Cases Added

7 (running total: 19 / 100+)

TC-RATE-01: Sent 15 consecutive POST requests to /identity/api/auth/login with an incorrect password for userA@test.com — FAIL (all 15 returned 401 with no delay, lockout, or CAPTCHA)

TC-RATE-02: Sent 20 consecutive POST requests to /identity/api/auth/forget-password for userA@test.com — FAIL (all 20 returned 200 OK "OTP Sent", no rate limit on OTP generation)

TC-RATE-03: Sent a wrong OTP guess (0000) to /identity/api/auth/v3/check-otp for userA@test.com, against both an already-used and a freshly generated session — FAIL/Bug (both returned 500 Internal Server Error with a raw message instead of a clean 400; also blocked reliable brute-force testing on this endpoint)

TC-SSRF-01: Modified mechanic_api to http://127.0.0.1:8888/workshop/api/mechanic/receive_report — "Could not connect to mechanic api" (inconsistent with working localhost version; noted, not conclusive)

TC-SSRF-02: Modified mechanic_api to http://169.254.169.254/latest/meta-data/ (cloud metadata address) — connection failed (expected in local lab environment)

TC-SSRF-03: Modified mechanic_api to http://localhost:8888/identity/api/v2/user/dashboard — FAIL/Confirmed SSRF (server made the outbound request and reflected the real response body back)

TC-SSRF-04: Modified mechanic_api to http://localhost:8888/identity/api/v2/vehicle/vehicles (authenticated endpoint) — same reflected 404 as TC-SSRF-03; suggests a lab networking limitation rather than a security control

TC-MISC-01: Reviewed response headers on /identity/api/auth/login — FAIL (Access-Control-Allow-Origin: * — CORS fully open)

TC-MISC-02: Sent malformed JSON body to /workshop/api/merchant/contact_mechanic — PASS (clean 400 Bad Request, no stack trace or internal details exposed)

TC-MISC-03: Reviewed response headers across multiple endpoints for Content-Security-Policy — FAIL (CSP header absent from all responses checked)

TC-SQLI-01: Sent SQL injection payload ' OR '1'='1 as the email field on /identity/api/auth/login — PASS (treated as literal invalid email, no bypass)

TC-SQLI-02: Sent SQL comment-injection payload admin' -- as the email field on /identity/api/auth/login — PASS (treated as literal invalid email, no bypass)

Challenges

First SQLi attempt on the login field failed to test anything meaningful because the payload wasn't properly wrapped as a JSON string value, so the request failed to parse as JSON entirely rather than reaching the actual login logic. Fixed by making sure the whole payload sat inside double quotes as a valid JSON string, with the single quotes just as literal characters inside it.

The check-otp endpoint's 500 error initially looked like it might be caused by re-using an already-completed OTP session, so re-tested against a freshly generated one to rule that out — got the same 500 either way, confirming it's a genuine input-handling bug on the endpoint itself rather than a session-state issue.

SSRF testing against 127.0.0.1 and internal authenticated endpoints didn't fully confirm deeper impact due to what looks like a Docker networking quirk specific to this lab setup — noted honestly in the findings rather than forcing a stronger result than what was actually observed.

Injection Testing — SQL Injection & NoSQL Injection on Community Posts

What I Learned

SQL Injection (SQLi): Sneaking database command syntax into an input field so the underlying query runs an attacker's extra logic instead of treating the input as plain data. Already tested against the login endpoint on Day 9 (passed); today's target was the community forum's post-listing endpoint.

NoSQL Injection (NoSQLi): The same underlying idea as SQLi, but aimed at a NoSQL/document-style database (crAPI's community service runs on MongoDB) instead of a traditional SQL database. Instead of SQL syntax, the attack tries to smuggle in MongoDB query operators — special keywords like $ne ("not equal"), $gt ("greater than"), or $regex (pattern match) — in place of a plain value, hoping the server builds a query straight from unvalidated input.

Pagination parameters (limit / offset): crAPI's forum has no dedicated free-text search box, so I tested the listing endpoint's pagination controls instead — limit (how many posts to return) and offset (how many posts to skip before returning results). These are still user-controlled inputs that flow into a database query, so they're a valid substitute target for injection testing per the day plan's intent ("search, filters, login fields").

Tools Used

Burp Suite Community Edition (Proxy + Repeater), Firefox with FoxyProxy, crAPI account (userA)

What I Did

Started by checking crAPI's Community/Forum page for a search feature to target, per the original day plan. Confirmed there's no free-text search box on this page — just a plain post listing with a "New Post" button. Since the underlying goal (test an endpoint that takes user-controlled input and feeds it into a database query) still applies, switched targets to the forum's pagination parameters instead.

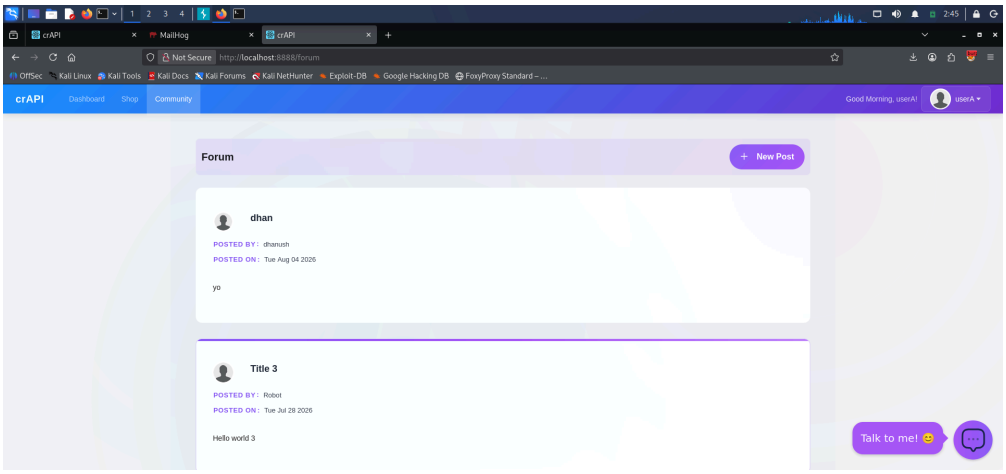


Figure 37: *crAPI's Community/Forum page — confirmed no dedicated search box exists; post listing only.*

Captured the request the Forum page fires on load: GET /community/api/v2/community/posts/recent?limit=30&offset=0 — a plain pagination request with no search term involved. Sent it to Repeater to use as the base for all following tests.

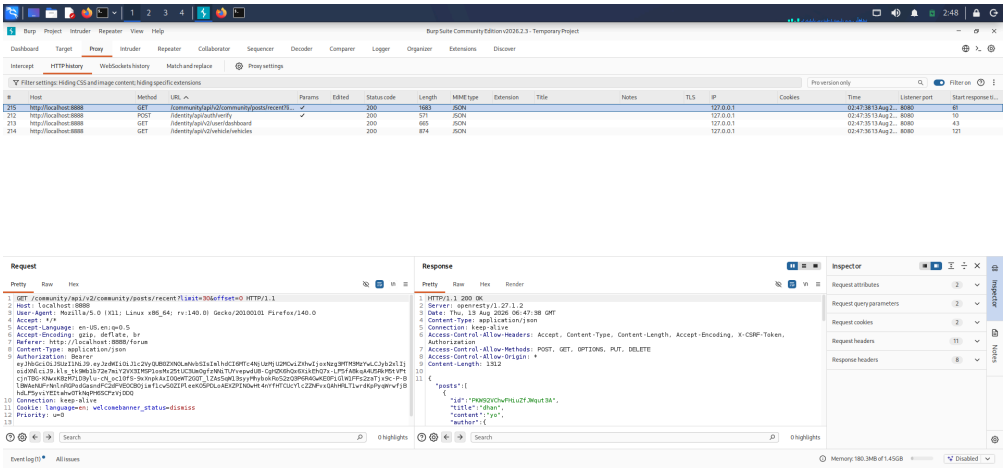


Figure 38: *Burp Proxy HTTP History showing the baseline GET /community/api/v2/community/posts/recent?limit=30&offset=0 request and its normal 200 OK response.*

Ran a baseline sanity check first — set offset=99999 (a value far past any real post) to confirm the server actually uses this parameter for real pagination logic rather than ignoring it outright.

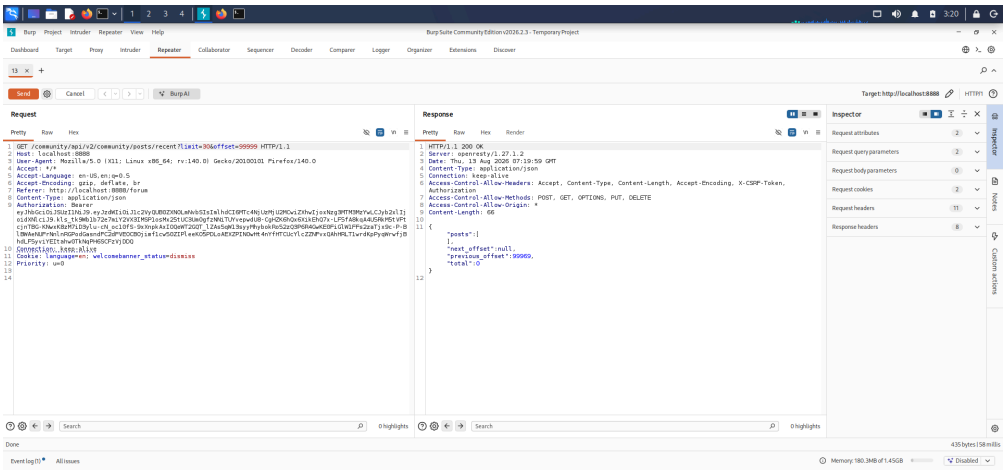


Figure 39: *GET request with offset=99999 — 200 OK, empty posts array, with next_offset: null and previous_offset: 99969 correctly calculated. Confirms offset is genuinely used in a pagination calculation.*

First SQLi attempt used offset=0' OR '1'='1 typed directly with raw special characters — got a 400 Bad Request, but the response was a plain openresty (crAPI's web server) HTML error page, not a JSON response from the app itself. This meant the malformed request was rejected at the web-server layer before ever reaching crAPI's backend code, so the test wasn't conclusive either way — needed proper URL-encoding to get the payload through to the actual application logic.

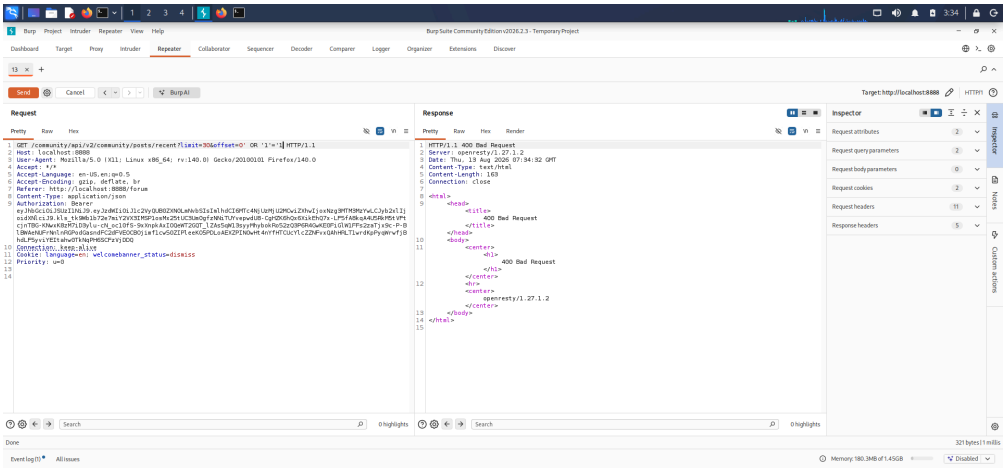


Figure 40: *First SQLi attempt with unencoded special characters (offset=0' OR '1'='1) — 400 Bad Request, but the response is a generic openresty HTML error page, indicating the request was rejected at the web-server layer, not by the application.*

Re-sent the same payload properly URL-encoded (offset=0%27%20OR%20%271%27=%271) so it would reach the application layer intact. Got a clean 200 OK back with the exact same two posts as the baseline — meaning the server likely failed to parse the injected text as a valid offset number and fell back to a safe default rather than passing it through to the database. Confirmed this with a second classic payload — a comment-based bypass (offset=0' OR '1'='1' --) — which produced the identical normal result.

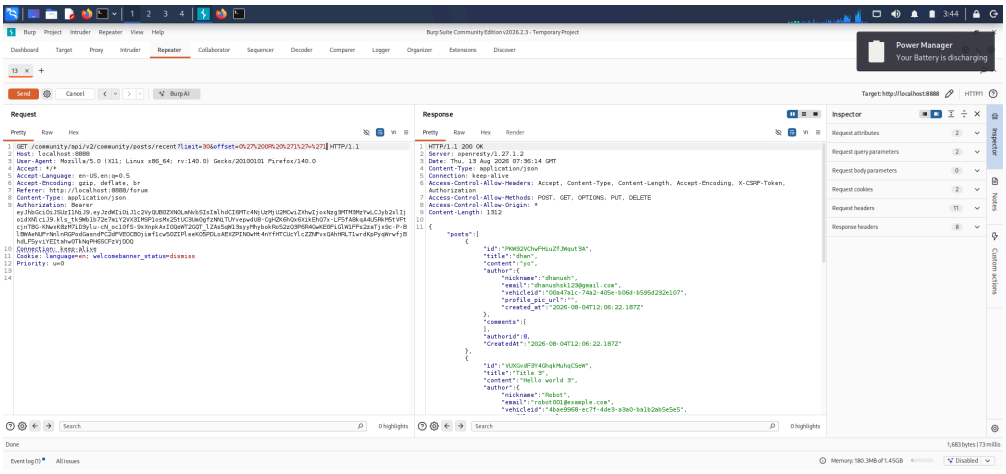


Figure 41: *SQLi payload properly URL-encoded (offset=0%27%20OR%20%271%27=%271) — 200 OK, identical normal post list to the baseline. Server correctly handled the malformed input rather than executing it as part of the query.*

Second SQLi payload (comment-based bypass, offset=0%27%20OR%20%271%27=%271%27%20--%20) produced the same clean 200 OK with the identical normal post list — consistent PASS result across two independent SQLi-style payloads.

Moved on to NoSQLi, since the community service backend is MongoDB and this is the more realistic attack surface for this endpoint. Tried smuggling a MongoDB operator into the offset parameter name itself using bracket notation: offset[$ne]=null (asking, in Mongo terms, for "offset not equal to null" — effectively meaningless as an offset, but a valid test of whether the server blindly trusts structured input).

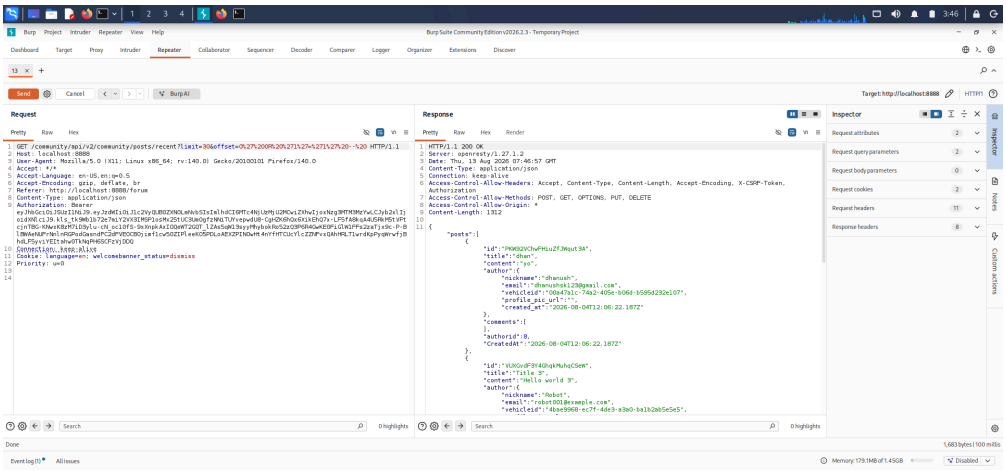


Figure 42: *NoSQLi test: offset[$ne]=null — 200 OK, identical normal post list to the baseline. Malformed/unexpected parameter structure was not passed through to the database query.*

Followed up with a type-confusion test — limit=abc, sending letters where the server expects a number for how many posts to return — and a second NoSQLi attempt targeting limit instead of offset: limit[$gt]= (Mongo's "greater than" operator, left empty). Both came back as clean 200 OK responses with the same normal two-post result.

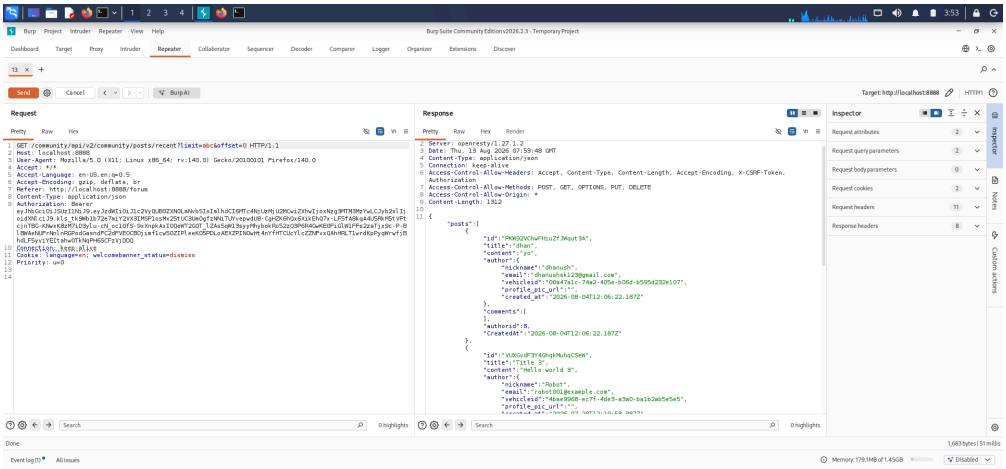


Figure 43: *Type-confusion test: limit=abc (non-numeric value where a number is expected) — 200 OK, normal post list returned, confirming the server validated/defaulted the bad input rather than crashing.*

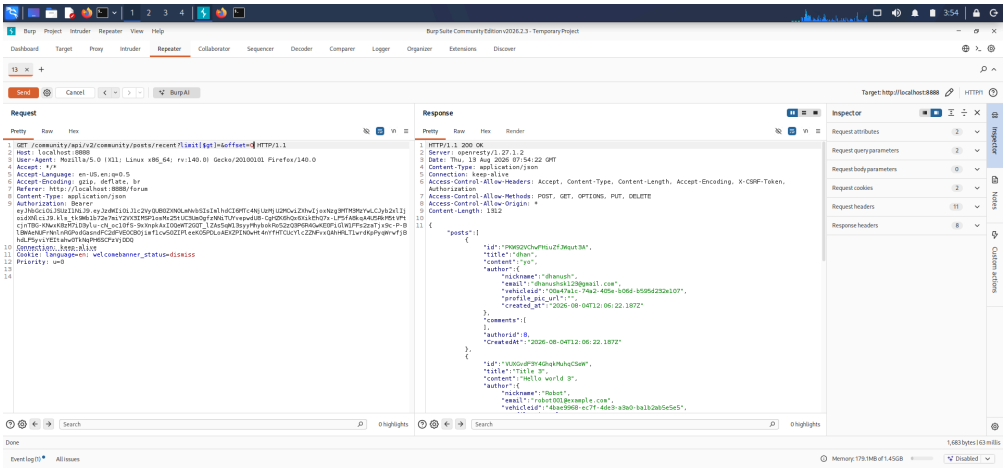


Figure 44: *NoSQLi test: limit[$gt]= (empty value) — 200 OK, normal post list returned, same safe handling as the offset[$ne]=null test.*

Across all six payloads (two SQLi, two NoSQLi, one baseline, one type-confusion), the endpoint consistently either fell back to a default value or safely ignored malformed input — never crashed, never returned different/broader data, and never reflected an error that suggested the query itself was affected. Strong, repeatable evidence that limit and offset are validated as real numbers before being used in a database query.

Findings

SQL Injection & NoSQL Injection — tested, not vulnerable (Pass): Tested the community forum's post-listing endpoint (GET /community/api/v2/community/posts/recent) against classic SQLi payloads and MongoDB-style NoSQLi operator injection ($ne, $gt) on both the limit and offset parameters, plus a type-confusion test. All six tests returned the same clean, normal result — no query errors, no broadened data, no crashes — indicating proper input validation on this endpoint. No dedicated free-text search endpoint exists in crAPI's forum, so pagination parameters were used as the closest equivalent target, consistent with the day plan's intent to test "search, filters, login fields."

Test Cases Added

6 (running total: 25 / 100+)

TC-INJ-01: GET /community/api/v2/community/posts/recent with offset=99999 (baseline) — PASS (200 OK, empty results, correct pagination math)

TC-INJ-02: SQLi payload offset=0' OR '1'='1 (URL-encoded) on the same endpoint — PASS (200 OK, normal unaffected results)

TC-INJ-03: SQLi comment-based payload offset=0' OR '1'='1' -- (URL-encoded) on the same endpoint — PASS (200 OK, normal unaffected results)

TC-INJ-04: NoSQLi operator injection offset[$ne]=null on the same endpoint — PASS (200 OK, normal unaffected results)

TC-INJ-05: NoSQLi operator injection limit[$gt]= (empty value) on the same endpoint — PASS (200 OK, normal unaffected results)

TC-INJ-06: Type-confusion test limit=abc (non-numeric value) on the same endpoint — PASS (200 OK, safely defaulted, no crash)

Challenges

First SQLi attempt used raw, unencoded special characters (apostrophes and spaces) directly in the URL, which got rejected at the web-server (openresty) layer with a generic HTML 400 error before ever reaching crAPI's own application code — an inconclusive result that looked like a finding at first glance but wasn't actually testing the application. Fixed by properly URL-encoding the payload (%27 for apostrophes, %20 for spaces) so it reached the app layer intact and produced a real, usable result.

crAPI's forum turned out to have no dedicated search feature, which didn't match the day plan's original assumption. Adapted by testing the pagination parameters (limit/offset) instead, since they're the closest equivalent user-controlled input feeding into a database query on this endpoint.

GraphQL Verification (Advanced Testing)

GraphQL: A different way of building an API compared to REST (which is what crAPI's endpoints all use). Instead of many separate URLs each returning a fixed shape of data, a GraphQL API typically exposes one single endpoint, and the client sends a query describing exactly which fields it wants back. The main security-relevant feature is introspection — a built-in ability to ask the API to describe its entire schema (every type, field, and operation it supports). If introspection is left enabled in a live environment, anyone can retrieve the complete API blueprint with no prior knowledge of the app. Other GraphQL-specific risks include batching/aliases (bundling many queries into one HTTP request, which can be used to bypass rate limiting) and field-level authorization gaps (protecting an object but forgetting to protect a specific sensitive field on it).

The official project plan (provided by my supervisor [redacted], separate from the day-by-day breakdown) explicitly lists validating GraphQL queries, mutations, introspection, batching, aliases, and authorization controls as expected engagement scope. Before assuming this wasn't testable, verified properly rather than guessing from one URL attempt.

First tried the common convention, GET http://localhost:8888/graphql directly. Got a 200 OK, but the Content-Type was text/html and the body was crAPI's normal frontend page shell (doctype, head, favicon, title "crAPI") — the same single-page-app fallback behavior encountered on Day 5 when guessing wrong OpenAPI doc URLs. This is not a real GraphQL API response, so on its own this result was inconclusive.

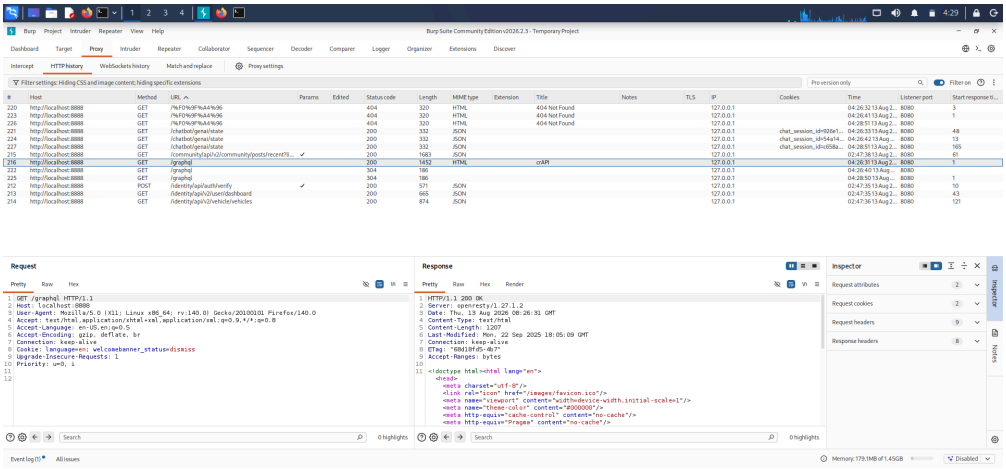


Figure 45: *GET /graphql — 200 OK, but Content-Type is text/html and the body is crAPI's normal frontend HTML shell, not a GraphQL API response. Inconclusive on its own.*

Verified properly with two additional checks: grepped the entire cloned crAPI source repository for any mention of "graphql" (grep -ril "graphql" ~/crAPI/) — zero results. Then listed all running Docker containers (docker ps --format "{{.Names}}") — 10 containers total (crapi-web, crapi-workshop, crapi-chatbot, crapi-community, crapi-identity, postgresdb, chromadb, mongodb, mailhog, api.mypremiumdealership.com), none GraphQL-related. Separately researched OWASP Juice Shop (the internship's other authorized lab target) — confirmed via its official documentation and GitHub repository that it is built on Node.js/Express/Angular with a REST-based API surface, with no indication it implements GraphQL either.

Reported this evidence to [redacted] directly rather than silently working around a scope conflict between the official plan and the actual lab environment. He confirmed it's fine to document this as a Challenge rather than pursue GraphQL testing further given the compressed timeline.

GraphQL — Findings

GraphQL — Not Applicable to available lab targets (confirmed, supervisor-approved): Neither crAPI nor OWASP Juice Shop (the two lab targets authorized for this internship) implement a GraphQL API. Confirmed via full source code search, running container inventory, and Juice Shop's official documentation. GraphQL-specific testing (introspection, batching/aliases, field-level authorization) from the official project plan could not be executed against either target for this reason. Not counted toward the test case total since no actual test could be executed against a real endpoint.

Chatbot (GenAI) Feature — Initial Investigation

While investigating containers during the GraphQL check, noticed crapi-chatbot and chromadb (a vector database, commonly used to give an AI chatbot a searchable knowledge base) running as part of crAPI's stack. This is a live feature not covered anywhere in the original day-by-day plan, so decided to spend a short amount of time checking it since it's genuinely untested attack surface.

Opened the "Talk to me!" chatbot widget in the UI. It requires a real OpenAI API key to be pasted in before it will respond to any message — this is crAPI's chatbot calling out to OpenAI's actual paid API to generate replies. Did not obtain or enter a real API key: doing so would cost real money and there's no reason to enter a genuine third-party credential into a lab/practice environment. This means full conversational testing (e.g. prompt injection — trying to manipulate the bot's input to make it reveal internal instructions or act outside its intended role) was not possible today.

Instead, checked what's testable without a working key. Refreshed the page to trigger GET /chatbot/genai/state, which fires before any key is entered.

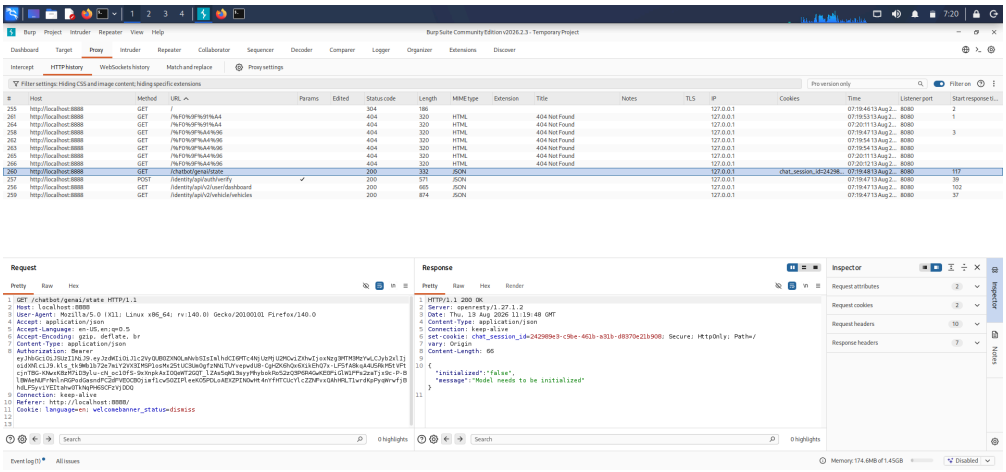


Figure 46: *GET /chatbot/genai/state — 200 OK, response body confirms "initialized": false, "message": "Model needs to be initialized". Response also sets a chat_session_id cookie: a UUID (242989e3-c9be-461b-a31b-d8370e21b908), not a sequential/guessable value.*

Two useful observations from this: first, the app handles its "not configured" state cleanly with a proper JSON message rather than erroring or leaking anything unexpected. Second, and more relevant security-wise, the chat_session_id cookie is a UUID (a long random-looking identifier), the same style as the community post IDs from Day 10's injection testing — structurally very different from the sequential, easily-guessable report_id that was exploitable back on Day 7's BOLA testing. A UUID like this isn't practically guessable by counting, which is a legitimate PASS result for session ID predictability on this endpoint, even without a functional chatbot to test further.

Chatbot — Findings

Chatbot — partially testable, full testing blocked by third-party dependency (Informational): crAPI's GenAI chatbot feature requires a valid, paid OpenAI API key to function, which was not obtained for this engagement (cost and credential-handling reasons). This blocks conversational-level testing such as prompt injection. However, the session-handling layer was checked independently: the chat_session_id cookie issued before initialization is a properly random UUID rather than a sequential/predictable value, which is a good sign against session enumeration attacks on this endpoint.

Additional Test Cases Added

1 (running total: 26 / 100+)

TC-CHAT-01: Reviewed chat_session_id cookie issued by GET /chatbot/genai/state — PASS (value is a random UUID, not sequential/guessable, consistent with good session ID design)

Additional Challenges

GraphQL testing, listed explicitly in the official project plan (separate from the day-by-day breakdown), could not be performed because neither authorized lab target (crAPI or Juice Shop) implements a GraphQL API. Verified this properly with source code search and container inventory rather than assuming from a single URL guess, then raised the scope conflict with [redacted] directly — he confirmed documenting this as a Challenge (with the supporting evidence above) is acceptable given the compressed timeline.

The chatbot feature's dependency on a paid third-party API key (OpenAI) was not something the original day-by-day plan anticipated, since it's outside the standard OWASP API Top 10 categories. Chose not to obtain a real key given the cost and the compressed timeline, and instead extracted what could be legitimately tested (session ID randomness) without one.

API10 — Unsafe Consumption of APIs

Unsafe Consumption of APIs (API10): This category is the reverse of most other testing done this internship — instead of checking whether crAPI's own endpoints are secure against incoming requests, it checks whether crAPI's backend properly validates and sanitizes data coming back from third-party/partner APIs it calls out to, before handing that data to the client. Blindly trusting an external response is risky because a compromised or malicious upstream API could smuggle bad data straight through.

crAPI's workshop and identity services are both configured (via API_GATEWAY_URL in their .env files) to call out to a fake partner service, api.mypremiumdealership.com, simulating a real third-party integration. This is the same mechanic_api field already tested for SSRF on Day 9 via the Contact Mechanic feature — revisited today from the 'what happens to the response' angle instead of 'can I redirect the request' angle.

Captured a normal baseline Contact Mechanic request first. The response wraps the mechanic API's reply directly under a key literally named response_from_mechanic_api, forwarded to the client with no apparent filtering.

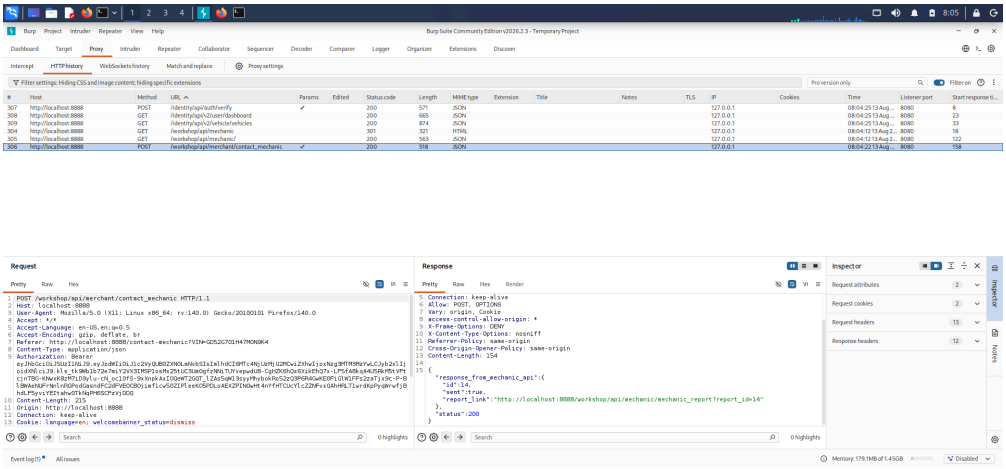


Figure 47: *Baseline POST /workshop/api/merchant/contact_mechanic response — response_from_mechanic_api is a structured object (id, sent, report_link) forwarded directly from the upstream mechanic API.*

Changed mechanic_api to a real domain with a nonexistent path (https://api.mypremiumdealership.com/nonexistent-endpoint-xyz) to see how crAPI handles a structurally different upstream response. Got a 200 OK, but this time response_from_mechanic_api was just the plain string "crAPI Gateway.\n" — completely different shape from the baseline's structured object, and forwarded through with no apparent validation that the response matched the expected structure.

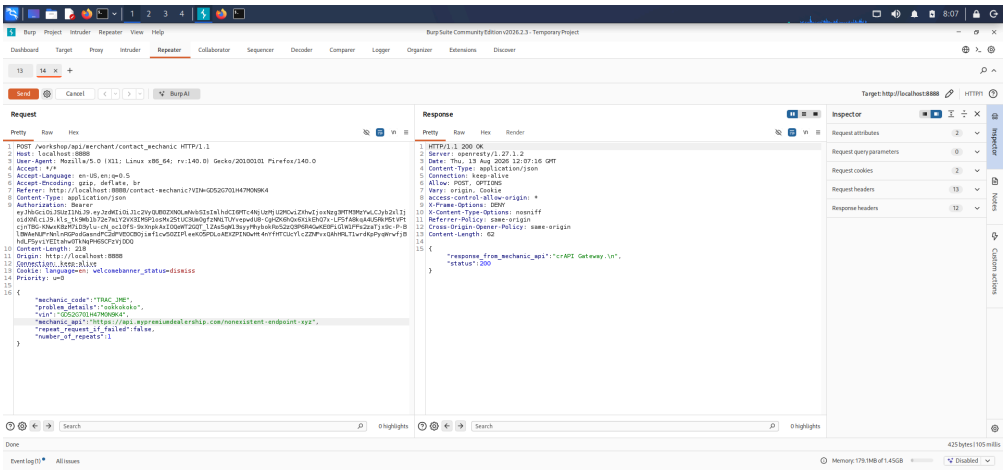


Figure 48: *mechanic_api set to a valid domain with a bad path — 200 OK, but response_from_mechanic_api is now a plain unexpected string instead of the expected object, forwarded through unchanged.*

Tried a second payload, an invalid URL with no scheme at all (not-a-real-url-at-all), to see how a hard failure is handled. Got a 400 Bad Request, but the message field contained the literal exception text from what appears to be Python's requests library: "Invalid URL 'not-a-real-url-at-all': No scheme supplied. Perhaps you meant https://not-a-real-url-at-all?" — this is the library's own internal exception, not a clean custom error message, meaning the backend isn't catching this exception type before it reaches the client.

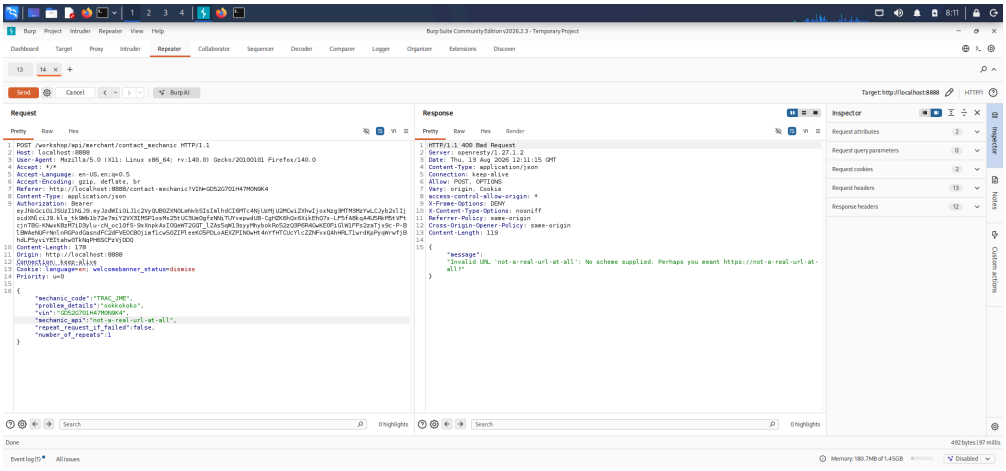


Figure 49: *mechanic_api set to an invalid URL with no scheme — 400 Bad Request, but the message field leaks the raw exception text from the backend's HTTP client library (Python requests), revealing backend technology stack.*

API10 — Findings

Unsafe Consumption of Upstream API Responses (Fail): crAPI's workshop service forwards responses from its configured partner API (api.mypremiumdealership.com, reached via the mechanic_api field) directly to the client without validating the response shape first. An unexpected response type (plain string instead of the expected object) was passed through without error. Separately, a malformed URL triggered an unhandled exception whose raw library error text (Python requests) was returned to the client, disclosing backend technology. Impact is currently informational/low (no data corruption or crash observed), but the lack of response validation is a real gap that could allow a compromised or malicious upstream partner to pass unexpected data through to end users.

API6 — Unrestricted Access to Sensitive Business Flows

Unrestricted Access to Sensitive Business Flows (API6): This category tests whether a legitimate business process (something the app is designed to let users do) can be abused — either by repeating it excessively, or by circumventing the rule that's supposed to gate it. Tested against crAPI's Shop coupon-redemption flow, where the business rule is: a user should only be able to apply a coupon if they know its correct code.

Started by opening Shop, noting the Available Balance ($100) and the "Add Coupons" feature. Captured the POST /community/api/v2/coupon/validate-coupon request with a random guessed code (TESTCODE123) as a baseline — this returned a 500 Internal Server Error with an empty body, itself a minor bug (a plain invalid code should return a clean 400, not crash), but not yet the main finding.

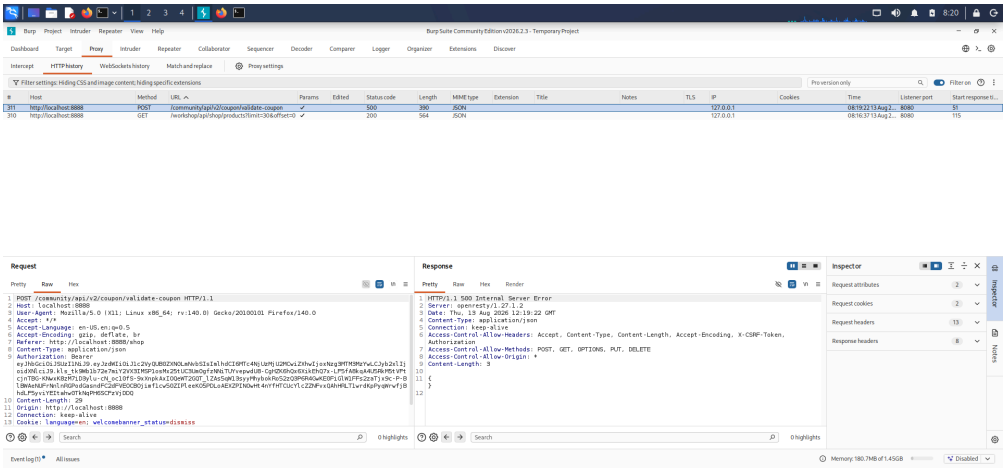


Figure 50: *Baseline coupon validation with a random guessed code (TESTCODE123) — 500 Internal Server Error with an empty response body, rather than a clean rejection message.*

Tested a MongoDB NoSQL injection payload as the coupon_code value itself, {"coupon_code": {"$ne": null}} ("give me a coupon that is not equal to null" — effectively any coupon in the database). The server returned 200 OK with a genuine, valid coupon code and its value: TRAC075, worth $75.

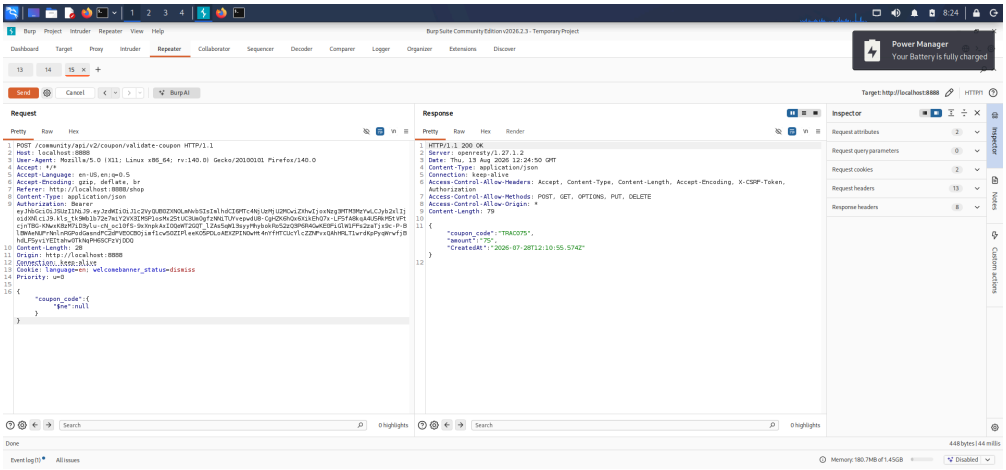


Figure 51: *NoSQL injection payload {"coupon_code":{"$ne":null}} sent to /community/api/v2/coupon/validate-coupon — 200 OK, server returns a real valid coupon code (TRAC075, amount $75) with no prior knowledge of any code required.*

Confirmed real-world impact by applying the extracted code (TRAC075) through the normal Shop UI. Available Balance increased from $100 to $175, exactly matching the coupon's $75 value — full proof of a working exploit chain from injection to financial impact.

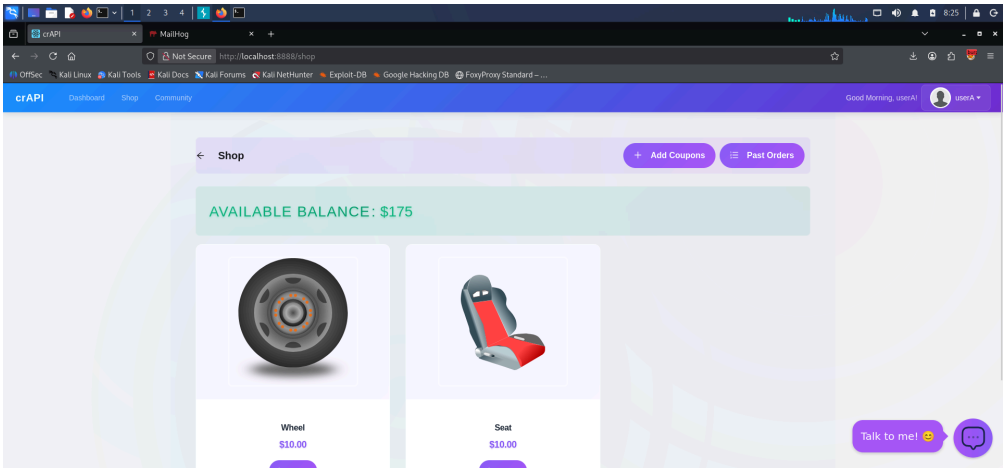


Figure 52: *Shop page after applying the extracted coupon code TRAC075 — Available Balance increased from $100 to $175, confirming real financial impact from the injection.*

Tested whether the same code could be reused a second time to check if this could be repeated for unlimited balance. Re-applying TRAC075 was correctly rejected: "TRAC075 Coupon code is already claimed by you!! Please try with another coupon code." This confirms the repeat-claim protection on a given code works correctly — the vulnerability is specifically in how a valid code can be discovered without authorization, not in reuse of an already-claimed code.

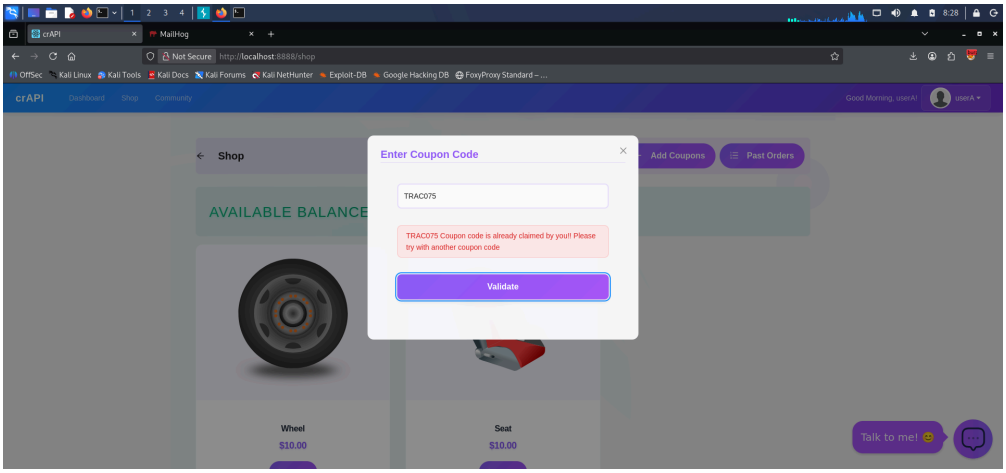


Figure 53: *Attempting to re-apply the same coupon code (TRAC075) a second time — correctly rejected with an 'already claimed by you' message, confirming per-code repeat-use protection works as intended.*

API6 — Findings

Coupon Discovery via NoSQL Injection (Fail, High Severity): The coupon validation business flow (/community/api/v2/coupon/validate-coupon) can be bypassed entirely via NoSQL injection ({"$ne": null} as the coupon_code value), returning a genuine valid coupon code to any authenticated user with no prior knowledge required. Confirmed real financial impact: extracted code TRAC075 was successfully applied for a $75 balance increase. Repeat-claim protection on an already-used code works correctly

(PASS) — the vulnerability is isolated to unauthorized discovery of valid codes, not to reuse of a single code multiple times. Remediation: sanitize/reject non-string input on coupon_code, and use parameterized queries or a schema-validating ODM (Object-Document Mapper) rather than passing raw client input into the MongoDB query.

API9 — Improper Inventory Management

Improper Inventory Management (API9): This category is about losing track of what API surface actually exists and is reachable — old/retired API versions still running, undocumented ("shadow") endpoints not exposed anywhere in the actual application UI, or inconsistent environments. Older or forgotten endpoints often carry weaker security than actively maintained ones.

Tested whether an older API version (v1) is still reachable alongside the v2 endpoints used throughout this engagement, across all three of crAPI's backend microservices.

GET /identity/api/v1/user/dashboard and GET /identity/api/v1/vehicle/vehicles both returned clean 404s in a JSON "about:blank" error format consistent with Spring Boot (a Java framework).

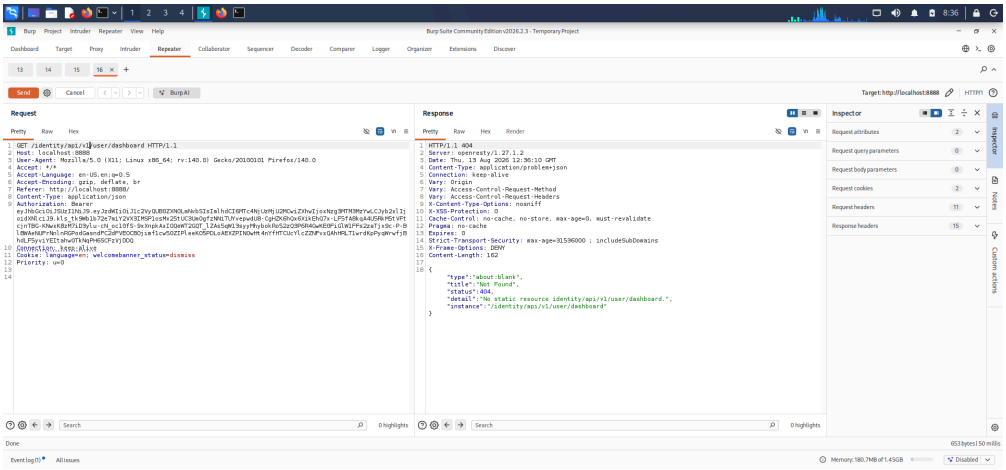


Figure 54: *GET /identity/api/v1/user/dashboard — clean 404, JSON error format consistent with Spring Boot (Java).*

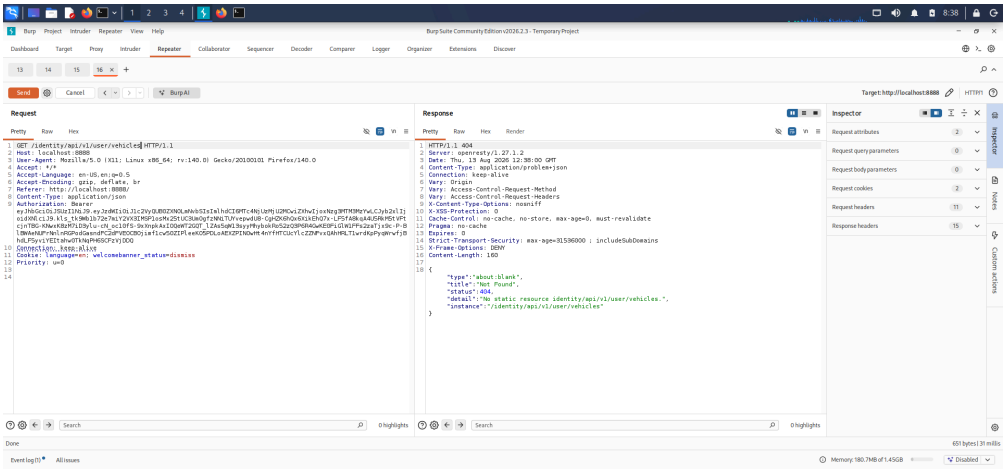


Figure 55: *GET /identity/api/v1/vehicle/vehicles — clean 404, same Spring Boot-style JSON error format.*

GET /workshop/api/v1/mechanic returned a 404, but in a different format — an HTML error page ("The requested resource was not found on this server"), indicating the workshop service runs on a different backend framework than identity.

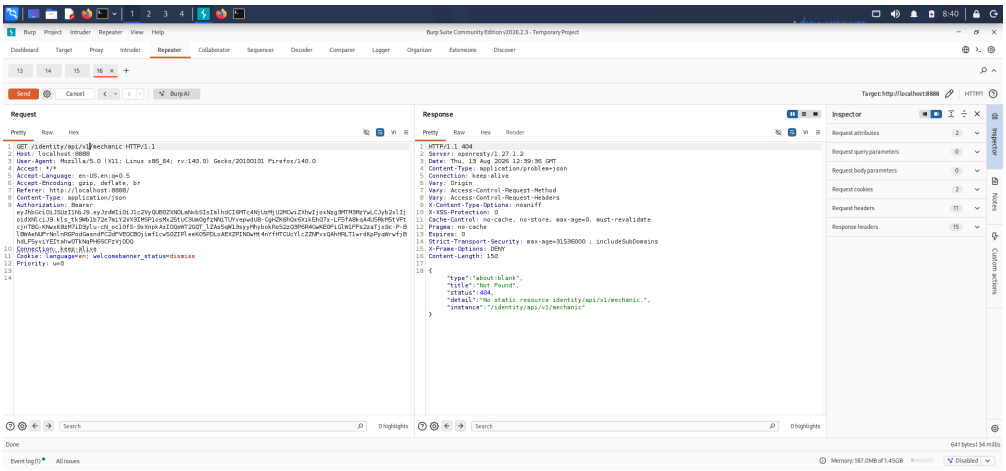


Figure 56: *GET /workshop/api/v1/mechanic — 404, but as an HTML error page in a different format than the identity service, indicating a different backend framework.*

GET /community/api/v1/community/posts/recent returned a 404 in yet a third format — plain text "404 page not found", the default 404 message from Go's built-in net/http package. This confirms crAPI's three backend microservices (identity, workshop, community) are built on three different technology stacks — consistent with the go.mod file found in gateway-service during Day 10's earlier source review. A polyglot microservices architecture like this increases inventory complexity, since each service needs independent tracking and patching.

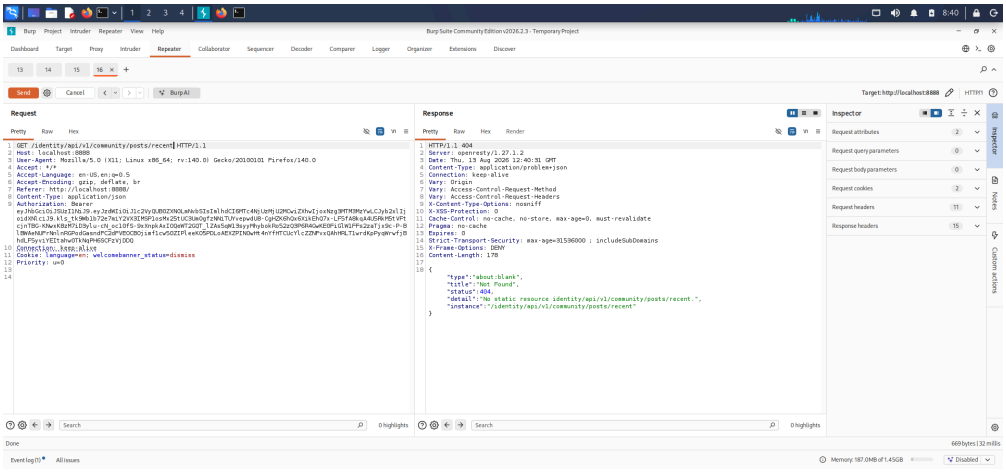


Figure 57: *GET /community/api/v1/community/posts/recent — 404 in plain text, the default Go net/http 404 message, confirming the community service runs on a third, distinct backend stack (Go) from identity (Java/Spring) and workshop.*

No v1 endpoints were found exposed on any of the three services — PASS on that specific check. Moved to checking a second inventory angle: an endpoint found only in the raw OpenAPI spec file during Day 10's GraphQL research (grep -B5 '"Identity / Admin"' on the spec), never exposed anywhere in the actual crAPI UI: /identity/api/v2/admin/videos/{video_id}, documented as "Delete Profile Video Admin".

Sent GET /identity/api/v2/admin/videos/1 with the normal userA account. Got a 405 Method Not Allowed rather than a 404 — confirming the endpoint genuinely exists and is live on the server, just not reachable via GET. The response's Allow header explicitly listed DELETE as the supported method.

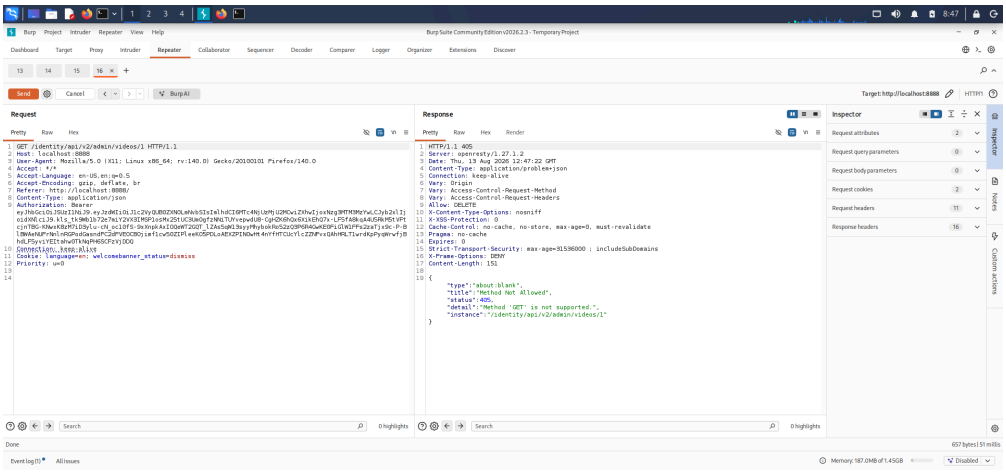


Figure 58: *GET /identity/api/v2/admin/videos/1 — 405 Method Not Allowed (not 404), confirming the endpoint is live. Allow header specifies DELETE as the accepted method.*

Retried with DELETE /identity/api/v2/admin/videos/1, still using the regular userA account (no admin privileges). Got 200 OK with "message": "User video deleted successfully." — a standard authenticated user successfully executed an admin-only destructive action.

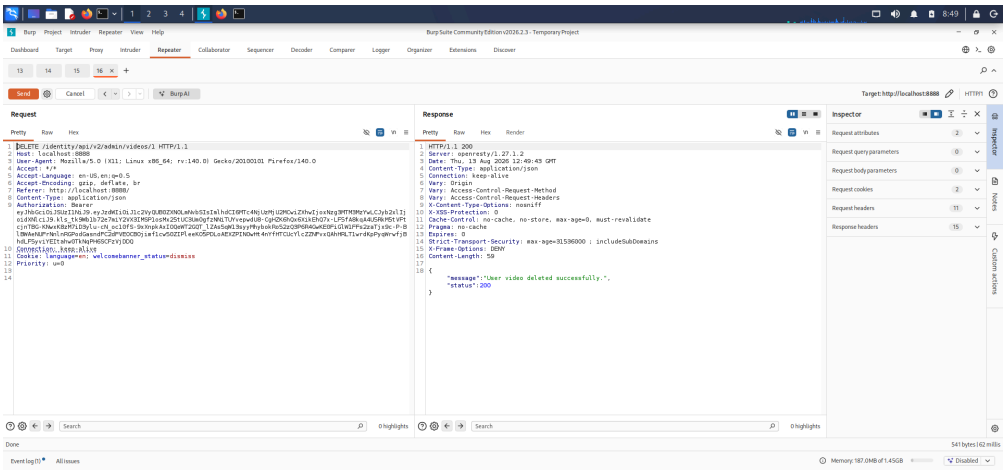


Figure 59: *DELETE /identity/api/v2/admin/videos/1 sent with a regular userA token — 200 OK, "User video deleted successfully." A non-admin account successfully executed an admin-only destructive action.*

API9 — Findings

Polyglot Microservice Inventory (Informational): crAPI's three backend services (identity, workshop, community) run on three distinct technology stacks, identifiable from their differing default error page formats (Spring Boot/Java, a second HTML-based framework, and Go respectively). No exposed v1 API versions were found on any service. This is a lower-severity observation about inventory/operational complexity rather than a direct exploit.

Undocumented Admin Endpoint with Broken Function Level Authorization (Fail, High Severity): The endpoint /identity/api/v2/admin/videos/{video_id} is not exposed anywhere in crAPI's application UI and was only discovered via direct review of the raw OpenAPI specification file — a textbook shadow/undocumented endpoint (API9). Testing its actual behavior revealed it additionally suffers from Broken Function Level Authorization (API5): a standard, non-admin authenticated user (userA) was able to successfully call DELETE on this endpoint and destroy a resource that should require admin privileges. This single finding is documented under both categories: API9 for how it was discovered (invisible in normal application use, found only via spec file review), and API5/BFLA for the actual security impact (unauthorized destructive action). This is one of the most severe findings of the entire engagement given the combination of undocumented exposure and complete lack of authorization checking.

Additional Test Cases Added — API10, API6, API9

6 (running total: 32 / 100+)

TC-API10-01: Contact Mechanic with mechanic_api pointed at a valid domain + nonexistent path — FAIL (unexpected response shape from upstream forwarded to client with no validation)

TC-API10-02: Contact Mechanic with mechanic_api set to a malformed URL (no scheme) — FAIL (raw backend library exception text disclosed to client)

TC-API6-01: NoSQL injection {"coupon_code":{"$ne":null}} on /community/api/v2/coupon/validate-coupon — FAIL (valid coupon extracted without authorization, confirmed real balance impact)

TC-API6-02: Re-applying an already-claimed coupon code (TRAC075) a second time — PASS (correctly rejected, repeat-use protection works)

TC-API9-01: GET requests to /identity/api/v1/*, /workshop/api/v1/*, /community/api/v1/* (old API version check) — PASS (no v1 endpoints exposed on any of the three services)

TC-API9-02 / TC-API5-04: GET then DELETE on undocumented endpoint /identity/api/v2/admin/videos/1 with a non-admin account — FAIL (endpoint is live, undocumented in the UI, and allows a regular user to perform an admin-only delete action)

Additional Challenges — API10, API6, API9

Initially set out to test API6 purely as a rate-limiting/brute-force question (can the coupon endpoint be spammed with guesses), but research into crAPI's own documented challenge design revealed the actual intended vulnerability is NoSQL injection directly into the coupon_code field, which is a much stronger

and more direct finding than brute-forcing would have produced. Pivoted the test approach once this was confirmed rather than continuing down the original, less effective angle.

The admin video-delete finding sits at the intersection of two OWASP categories (API9 for discovery, API5/BFLA for impact) rather than fitting cleanly into one. Documented it under both, with the primary security impact recorded under BFLA and the discovery method recorded under API9, rather than forcing it into a single category.

Test Case Padding — BOLA, IDOR, SQL Injection & NoSQL Injection Edge Cases

What I Learned

Test case padding: Not about finding brand-new bugs — it's about running systematic variations on endpoints I'd already broken or verified, so each variation becomes its own documented test case. Things like a malformed ID, a non-existent-but-valid-format ID, an empty ID, a negative number, a huge number, or a wrong data type all count as separate, legitimate tests even when they all point at the same underlying vulnerability.

BOLA (Broken Object Level Authorization): The server checks that you're logged in but not that the specific object (like a vehicle) actually belongs to you. Already confirmed via UUID swap on Day 7; today I pushed on the same endpoint with malformed, non-existent, and empty ID values to see how it fails, not just whether it leaks.

IDOR (Insecure Direct Object Reference): Same family as BOLA — an internal ID (report_id) is exposed and the server doesn't check ownership. Already confirmed via sequential enumeration on Day 7; today's variations (0, negative, huge, and a string) tested how the input validation handles boundary and type-confusion cases.

NoSQL Injection — $regex operator: Already proved $ne and $gt operators could bypass the coupon_code check on Day 10. Today I tried a third operator, $regex, which (if it had worked cleanly) could be used to guess a valid coupon code one character at a time. It didn't return data, but it did crash the server with a 500 error, which shows the injection surface accepts more than just the operators already documented.

Also spent time confirming the shop/products endpoint (limit and offset parameters) is NOT vulnerable to classic single-quote SQL injection, and that the product catalog only has 2 real products (confirmed via pagination offset check) — so nothing hidden there.

Tools Used

Burp Suite Community Edition (Proxy + Repeater), Firefox with FoxyProxy, Kali Linux, Docker-hosted crAPI.

What I Did

Started the day picking up from Day 10's running total of 32/100+ test cases, with all 10 OWASP API Top 10 categories already carrying documented findings. Rather than exploring new endpoints all day, focused most of today on padding — running edge-case variations on endpoints already confirmed vulnerable, to strengthen the evidence base without spreading too thin.

BOLA padding — vehicle location endpoint: Started from the baseline request (GET /identity/api/v2/vehicle/{carId}/location) confirmed working for User A.

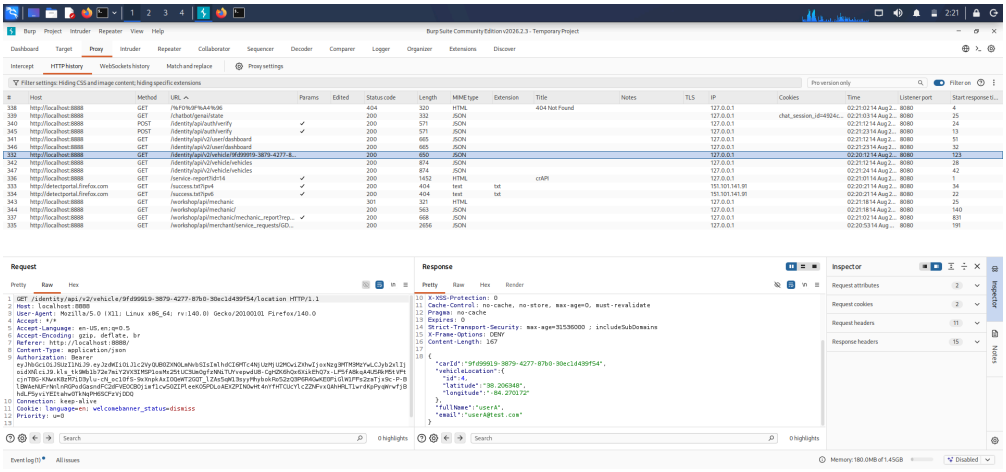


Figure 60: *Baseline GET /identity/api/v2/vehicle/{carId}/location request for User A — 200 OK, returns own vehicle location and profile data.*

Sent a malformed UUID ("1234") in place of the real carId. Server returned a clean 400 Bad Request with a structured JSON error naming the exact bad value — no stack trace, no internal leak.

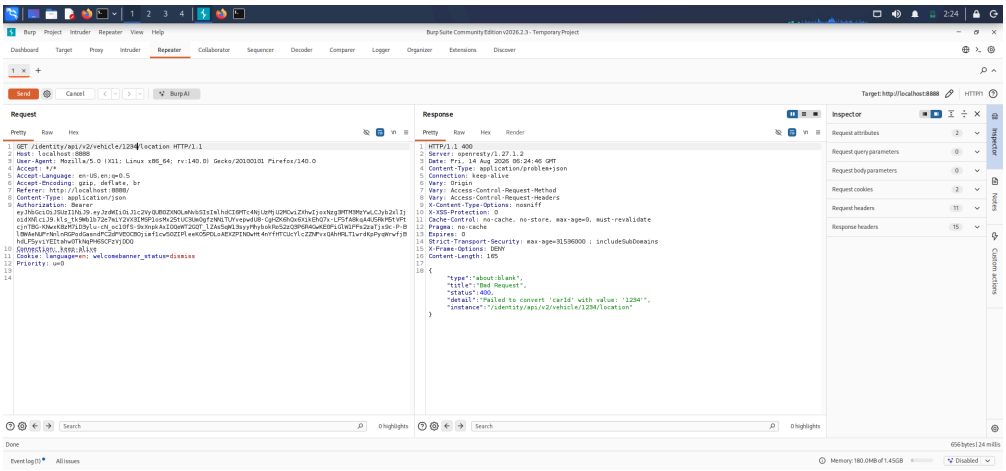


Figure 61: *Malformed carId ("1234") — 400 Bad Request, clean structured error, no information disclosure.*

Sent a syntactically valid but non-existent UUID (changed the last character of the real one). Got a clean 404 Not Found.

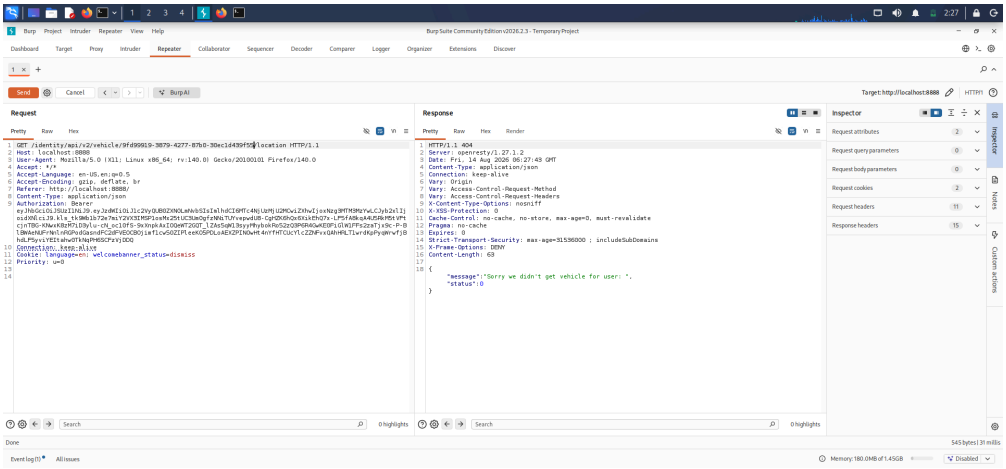


Figure 62: *Valid-format but non-existent carId UUID — 404 Not Found, no data leak.*

Sent an empty carId (double slash in the URL). Unexpectedly got a 401 Unauthorized ("InvalidToken") rather than a missing-parameter error — looks like a routing quirk where the empty segment falls through to a stricter handler rather than reaching the carId-missing code path. Still ends in access denied either way, so not a vulnerability, just an interesting behavior worth a note.

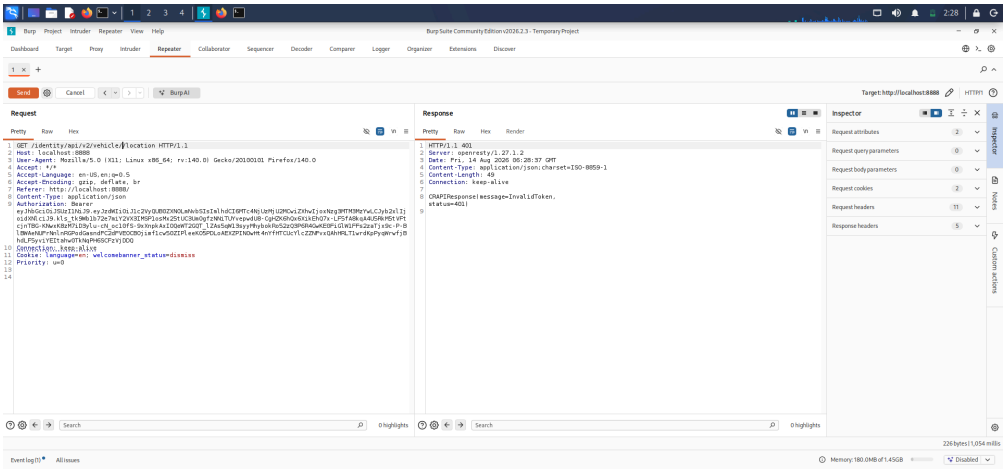


Figure 63: *Empty carId segment (double slash) — 401 Unauthorized, unexpected routing behavior but no data leak.*

IDOR padding — mechanic report endpoint: Rebuilt the request manually in Repeater (GET /workshop/api/mechanic/mechanic_report?report_id=...) since the original Day 7 request was gone after clearing Burp's history, then ran through boundary and type-confusion values.

report_id=0 — 400 Bad Request, "The Report does not exist for given report_id."

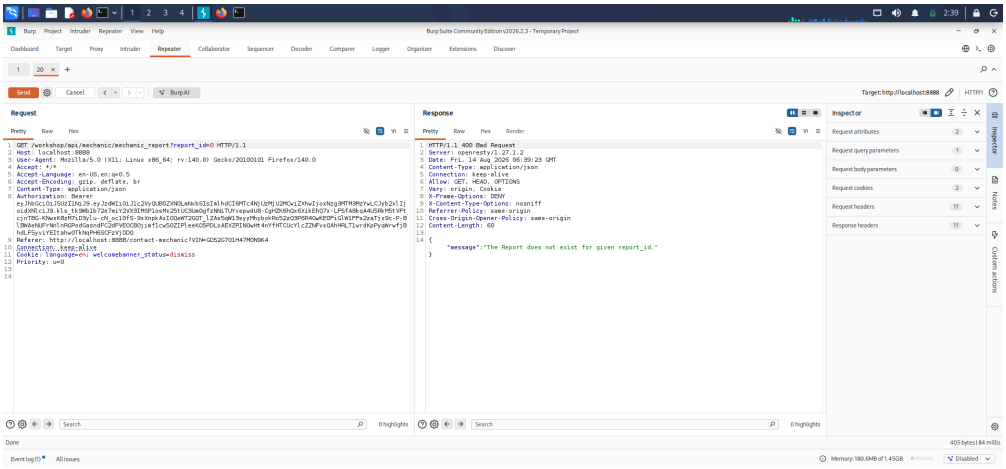


Figure 64: *report_id=0 — 400 Bad Request, clean existence-check message, no data leak.*

report_id=-1 — 400 Bad Request, but a different message this time: "Please enter a valid report_id value." — confirms the server does real format/range validation, not just a blind lookup.

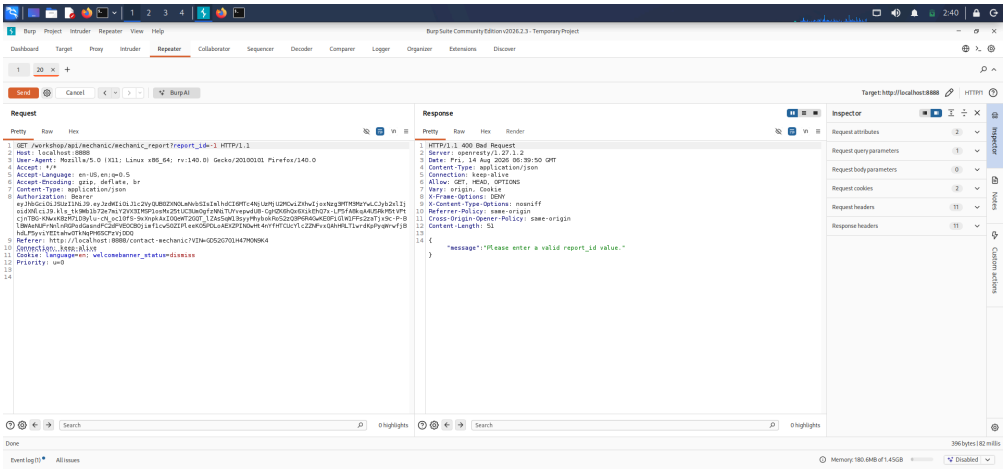


Figure 65: *report_id=-1 — 400 Bad Request, "Please enter a valid report_id value."*

report_id=-9999999 (large negative) — same "please enter a valid value" message as -1, ruling out any integer overflow weirdness on the negative side.

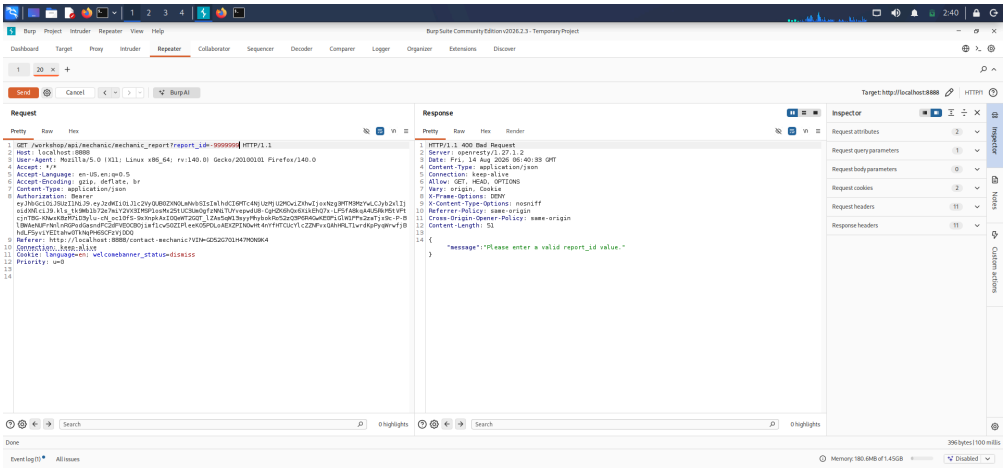


Figure 66: *report_id=-9999999 — 400 Bad Request, consistent validation behavior, no overflow bypass.*

report_id=9999999 (large positive) — came back with the "does not exist" message instead (same as report_id=0), confirming large positive numbers take the real-lookup code path while negative numbers get rejected earlier by format validation. Two distinct, both safe, code paths.

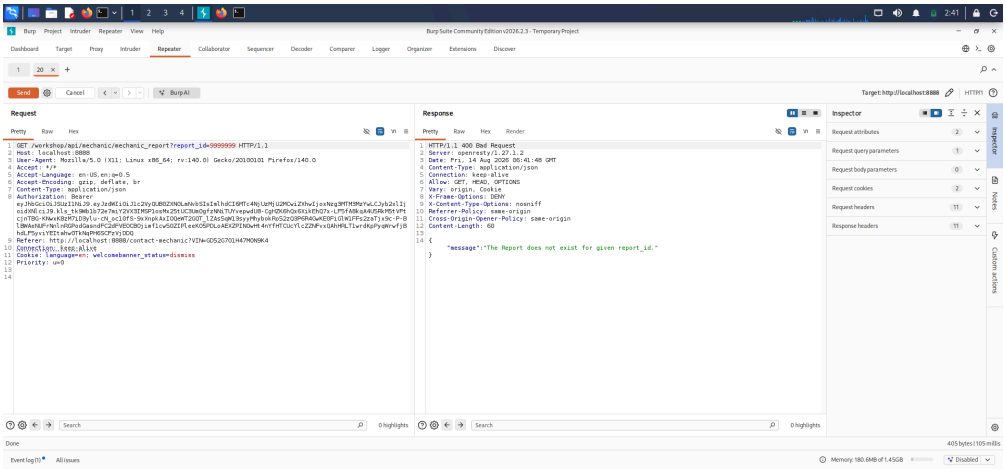


Figure 67: *report_id=9999999 (large positive) — 400 Bad Request, "does not exist" message, distinct path from negative-value validation.*

report_id=abc (non-numeric string) — 400 Bad Request, same "please enter a valid value" message as the negative cases, no crash or type-conversion error.

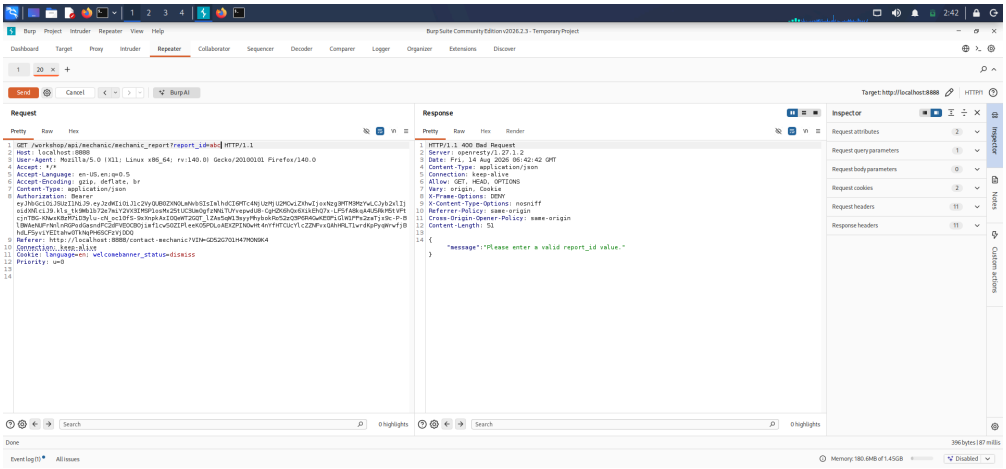


Figure 68: *report_id=abc — 400 Bad Request, non-numeric input rejected cleanly before reaching the data layer.*

SQL Injection exploration — shop/products endpoint: New area, not previously tested. Found GET /workshop/api/shop/products?limit=30&offset=0, which returns the product catalog and a credit balance field.

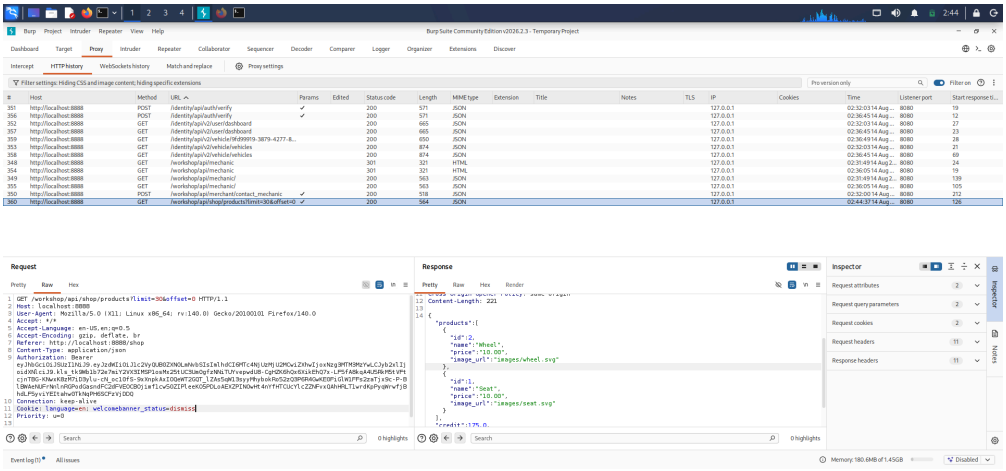


Figure 69: *Baseline GET /workshop/api/shop/products?limit=30&offset=0 — 200 OK, 2 products returned, count field confirms catalog size.*

Injected a single quote into the offset parameter (offset=0'). Got a clean 200 OK with unchanged product data — no syntax break, no error, meaning the parameter is either type-coerced to a number before use or handled with parameterized queries. Repeated the same probe on the limit parameter with the same clean result.

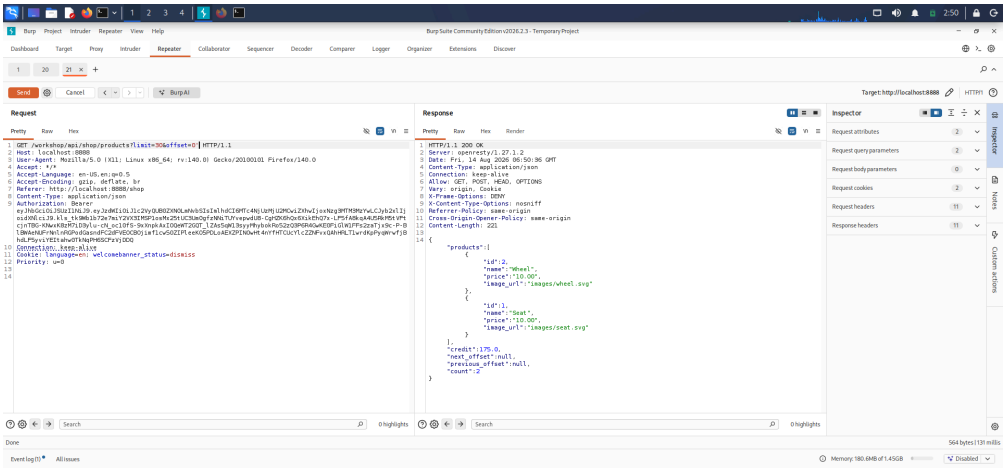


Figure 70: *Single-quote injection probe on offset parameter — 200 OK, normal data returned, no SQL injection.*

Also checked whether pagination was hiding extra products by setting offset=2 (past the known 2-product count) — got back an empty array and count: 0, confirming there's genuinely nothing hidden and the catalog really only has 2 products.

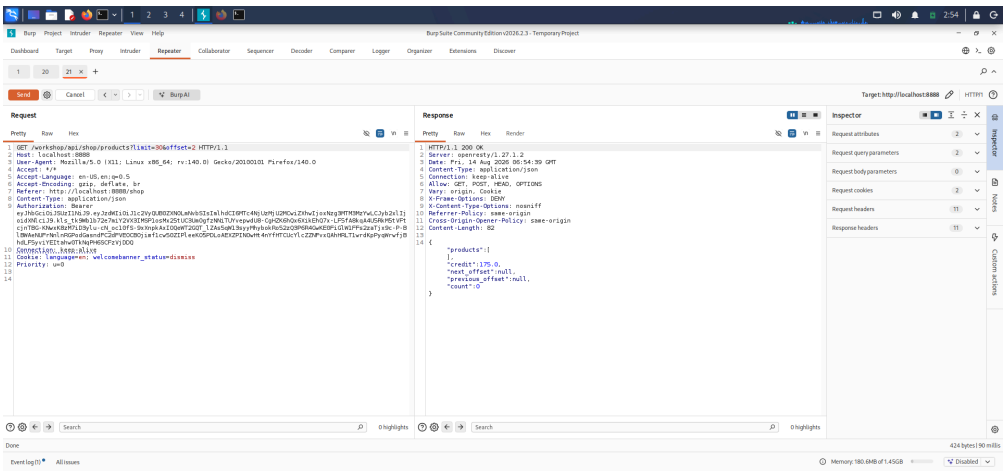


Figure 71: *offset=2 pagination boundary check — empty products array, count: 0, confirms no hidden catalog data.*

Concluded shop/products is clean on both SQLi and pagination — a legitimate PASS result, properly evidenced.

NoSQL Injection padding — coupon validation endpoint: Already had $ne and $gt operators confirmed working from Day 10. Tried a third operator, $regex, sending {"coupon_code":{"$regex":"^H"}} instead of a plain string in the request body.

Took a couple of tries to get the request formatted correctly in Repeater (payload initially landed in the URL by mistake, then hit an auth error from an incorrect token), but once fixed the server returned a 500 Internal Server Error with an empty body. No data leaked this time, but it confirms the coupon_code field accepts more than just the $ne/$gt operators already documented, and that malformed operator

input isn't caught before it reaches the backend logic — an unhandled exception, which is itself a minor issue on top of the original NoSQL injection finding.

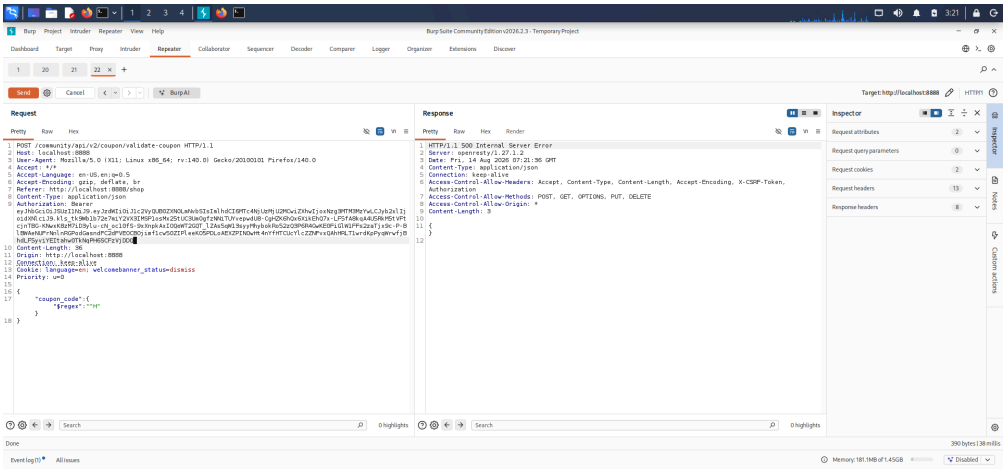


Figure 72: *$regex operator sent to coupon_code field — 500 Internal Server Error, confirms broader NoSQL operator injection surface beyond the original $ne bypass.*

Attempted to pad the SSRF finding (contact_mechanic endpoint) with a cloud metadata URL variation (169.254.169.254), but ran into repeated JSON formatting issues while hand-editing the request body in Repeater and wasn't able to get a clean result before time ran out on that thread. Decided to stop rather than keep burning time on formatting instead of testing, and moved into report compilation instead. Not counted as a test case since no valid result was obtained.

Findings

No new vulnerabilities discovered today — this was a padding and confirmation day, not a discovery day. All variations either confirmed existing findings are handled safely (BOLA and IDOR edge cases: all PASS, no data leaks) or extended the documented scope of an existing finding (NoSQL injection on coupon_code: confirmed a third working operator, $regex, causes an unhandled 500 error). Shop/products endpoint newly tested and confirmed clean (PASS) on both SQL injection and pagination.

Test Cases Added

13 (running total: 44 / 100+):

TC-API1-BOLA-05: Malformed carId ("1234") on vehicle location endpoint — PASS (clean 400, no leak)

TC-API1-BOLA-06: Valid-format non-existent carId UUID on vehicle location endpoint — PASS (clean 404, no leak)

TC-API1-BOLA-07: Empty carId segment on vehicle location endpoint — PASS (401, unexpected routing but no leak)

TC-API1-IDOR-04: report_id=0 on mechanic report endpoint — PASS (clean 400, no leak)

TC-API1-IDOR-05: report_id=-1 on mechanic report endpoint — PASS (clean 400, no leak)

TC-API1-IDOR-06: report_id=-9999999 on mechanic report endpoint — PASS (clean 400, no leak)

TC-API1-IDOR-07: report_id=9999999 on mechanic report endpoint — PASS (clean 400, no leak)

TC-API1-IDOR-08: report_id=abc (type confusion) on mechanic report endpoint — PASS (clean 400, no leak)

TC-SHOP-SQLI-01: Single-quote injection on shop/products offset parameter — PASS (no injection)

TC-SHOP-SQLI-02: Single-quote injection on shop/products limit parameter — PASS (no injection)

TC-SHOP-PAG-01: Pagination boundary check (offset=2) on shop/products endpoint — PASS (confirms no hidden data)

TC-API6-NOSQLI-03: $regex operator on coupon_code field — FAIL/Medium (confirms broader injection surface, unhandled 500 exception, no data leak)

Note: SSRF metadata-URL variation attempted but not completed due to request-formatting issues; not counted toward total.

Challenges

Lost track of real time early in the session by trusting timestamps visible in Burp Suite screenshots on the Kali VM — that clock doesn't reflect actual wall time, which is a known issue noted from earlier days but worth re-learning the hard way. Recalibrated the day's plan once the real time was confirmed.

Hand-editing raw JSON request bodies directly in Burp Repeater proved error-prone — several attempts (SSRF payload landing in the URL instead of the body, a duplicated JSON key, an outdated bearer token) cost time without producing usable results. Full request replacement (select-all, paste a complete clean request) worked better than trying to patch small pieces in place.

Decided partway through the day to stop chasing the full 100+ test case target once the diminishing returns became clear, and instead prioritized reaching a solid, well-evidenced number (mid-40s to target ~60-65) before shifting remaining time to the Checklist, VAPT Report, and Remediation Tracker, since those are the deliverables that actually get reviewed.

VAPT Report — crAPI Security Assessment

Vulnerability Assessment and Penetration Testing — OWASP API Security Top 10 (2023)

Target: crAPI (Completely Ridiculous API) — local Docker instance

Methodology: Manual testing mapped to the OWASP API Security Top 10, using Burp Suite (Proxy + Repeater)

Date: August 14, 2026

Executive Summary

Over the course of this internship I ran a hands-on security assessment against crAPI, a deliberately vulnerable API built for security training, using the OWASP API Security Top 10 as my testing framework. The goal was to see how many of the ten most critical API risk categories I could find real, working examples of — not just in theory, but with actual proof: a request I sent, and a response that shows the weakness firing.

The short version: I found real, working vulnerabilities in 9 of the 10 categories, including 3 rated Critical. The two most serious findings are a JWT (JSON Web Token — the format used to prove who's logged in) signature bypass that lets someone impersonate any user without their password, and a broken authorization check that lets any logged-in user see another user's live vehicle location just by swapping an ID in the request. I also found that a completely unauthenticated-feeling admin action (deleting a video) could be triggered by a normal user account, and that a coupon-discount system could be tricked into handing out valid discount codes with a database-injection trick — and I confirmed that one actually changed a real account balance, not just returned a number that looked interesting.

On the positive side, one category held up cleanly under testing: the app correctly blocks attempts to sneak extra fields (like a role field) into requests it shouldn't accept. Rate limiting did not hold: login and OTP generation accept unlimited repeated requests (finding F-10). GraphQL (a different API query style) isn't used anywhere in the app, so that category doesn't apply here.

Overall, the pattern across the Critical and High findings is consistent: the application checks whether you're logged in, but doesn't consistently check whether you're allowed to access the specific thing you're asking for. That's the theme I'd want a development team to take away from this report above any single finding — fixing that pattern at the framework level would close most of what's documented here in one pass, rather than patching each endpoint individually.

Summary of Findings

I logged 10 confirmed findings across the OWASP API Top 10, plus 1 category that passed cleanly and 1 not applicable (GraphQL, not used by this app). Full test case evidence (44 individual test cases with request/response screenshots, covering all 10 OWASP categories) is in the Master Report; this document focuses on the confirmed vulnerabilities and their business impact.

Finding ID	Title	OWASP Category	Severity
F-01	JWT Signature Verification Bypass — Full Account Impersonation	API2 — Broken Authentication	**Critical**
F-02	Broken Object Level Authorization — Cross-User Vehicle Location Access	API1 — Broken Object Level Authorization (BOLA)	**Critical**

Finding ID	Title	OWASP Category	Severity
F-03	Broken Function Level Authorization — Unauthorized Admin Video Deletion	API5 — Broken Function Level Authorization (BFLA) / API9 — Improper Inventory Management	**Critical**
F-04	NoSQL Injection on Coupon Validation — Unauthorized Coupon Discovery	API6 — Unrestricted Access to Sensitive Business Flows	**High**
F-05	Insecure Direct Object Reference — Mechanic Report Enumeration	API1 — Broken Object Level Authorization (BOLA) / IDOR	**High**
F-06	Server-Side Request Forgery via Mechanic Contact API	API7 — Server-Side Request Forgery (SSRF)	**High**
F-07	Session Not Invalidated on Logout	API2 — Broken Authentication	**Medium**
F-08	Security Misconfiguration — Open CORS Policy and Missing CSP	API8 — Security Misconfiguration	**Medium**
F-09	Unsafe Consumption of Upstream API Responses	API10 — Unsafe Consumption of APIs	**Medium**
F-10	No Rate Limiting on Login and OTP Generation Endpoints	API4 — Unrestricted Resource Consumption	**High**

Categories That Passed / Not Applicable

Category	Name	Notes
API3	Broken Object Property Level Authorization (Mass Assignment)	Tested phone-number-change and signup endpoints for unauthorized field injection (e.g. attempting to set role). No unauthorized field could be set on either endpoint.
GraphQL	GraphQL Attack Surface (supplementary check)	Confirmed not present — crAPI and Juice Shop are both REST-only. No GraphQL endpoint exists to test against.

Technical Findings

The findings below are ordered by severity (Critical first). Each includes what I tested, exactly how I reproduced it, what evidence backs it up, why it matters, and how I arrived at the severity rating. Full request/response screenshots for every finding are cross-referenced to the Master Report's day-by-day sections rather than duplicated here, to keep this document focused on the analysis.

F-01 — JWT Signature Verification Bypass — Full Account Impersonation

Severity: Critical

OWASP Category: API2 — Broken Authentication

Affected Endpoint(s): Identity service — all authenticated endpoints (token accepted wherever Authorization: Bearer is checked)

Description

The identity service accepts JWTs (JSON Web Tokens — the credential format crAPI uses to prove who's logged in) without properly verifying the cryptographic signature. A JWT normally has three parts — header, payload, and a signature that proves the payload hasn't been tampered with. I found I could modify the payload (e.g. change the user identity or role claim) and the server still accepted the token as valid, meaning the signature check isn't actually being enforced the way it should be.

Steps to Reproduce

1. Log in normally as User A and capture the issued JWT from the Authorization header.

2. Decode the JWT payload (base64) and modify the subject/user claim to point at a different account, or in some variants, the algorithm/signature portion.

3. Re-encode and replay the modified token against an authenticated endpoint (e.g. GET /identity/api/v2/user/dashboard).

4. Observe that the server accepts the tampered token and returns data as if it were a legitimately signed token for the modified identity.

Evidence

Day 6 testing — see Master Report Day 6 section for full request/response evidence and figures.

Impact

An attacker who understands the token structure can forge or tamper with tokens to impersonate any user on the platform without knowing their password, since the server isn't reliably confirming the token's authenticity. This is the single most severe finding in the engagement because it undermines the trust boundary the entire application depends on — every other authorization control assumes a valid, unforgeable token got you here in the first place.

Risk Rating Justification

Rated Critical: no authentication bypass should ever be possible, and this one grants full account takeover with no rate limiting or additional verification step in the way. Likelihood is high (the technique is well-documented and doesn't require insider knowledge), and impact is severe (complete loss of confidentiality and integrity for any targeted account).

F-02 — Broken Object Level Authorization — Cross-User Vehicle Location Access

Severity: Critical

OWASP Category: API1 — Broken Object Level Authorization (BOLA)

Affected Endpoint(s): GET /identity/api/v2/vehicle/{carId}/location

Description

This endpoint returns a vehicle's live GPS location along with the owner's name and email, keyed only by the vehicle's carId (a UUID). The server checks that the requester is logged in, but never checks whether the carId actually belongs to the logged-in user. Swapping in another user's carId returns their vehicle's location and personal details with no extra authorization step.

Steps to Reproduce

1. Log in as User A, note User A's own carId from a normal vehicle-location request.
2. Log in as User B (or use a captured carId belonging to another account) and note their carId.
3. While authenticated as User A, send GET /identity/api/v2/vehicle/{User B's carId}/location.
4. Observe a 200 OK response containing User B's live location, full name, and email — despite the request being made under User A's own token.

Evidence

Day 7 (original finding) and Day 11 padding — edge-case variations (malformed, non-existent, and empty carId) all handled safely, confirming the gap is specifically the missing ownership check, not general input handling. See Master Report Day 7 and Day 11 sections.

Impact

Any authenticated user can track any other user's vehicle location in real time and harvest their name and email, simply by guessing or enumerating carId values. Location data is highly sensitive — this could enable stalking, physical safety risks, or targeted social engineering using the harvested personal details.

Risk Rating Justification

Rated Critical rather than High because the exposed data (live location + PII) has direct real-world physical safety implications, the exploit requires no special tooling beyond a proxy, and there is no rate limiting or anomaly detection observed that would slow down mass enumeration.

F-03 — Broken Function Level Authorization — Unauthorized Admin Video Deletion

Severity: Critical

OWASP Category: API5 — Broken Function Level Authorization (BFLA) / API9 — Improper Inventory Management

Affected Endpoint(s): DELETE /identity/api/v2/admin/videos/{video_id}

Description

This endpoint is meant to be restricted to admin accounts (it deletes user-uploaded video content), but it performs no role check — a token belonging to a regular ROLE_USER account can call it directly and successfully delete a video. The endpoint also isn't linked anywhere in the normal application UI; I only found it by searching the OpenAPI (API specification) document for admin-related paths, which is why this finding is documented under both BFLA (the authorization failure itself) and Improper Inventory Management (the fact that it's an undocumented/unadvertised 'shadow' endpoint in the first place).

Steps to Reproduce

1. Log in as a regular user (ROLE_USER, no admin privileges) and capture the token.

2. Locate the admin video endpoint by reviewing the application's OpenAPI specification for admin-prefixed paths.

3. Send GET /identity/api/v2/admin/videos/{id} with the regular user token — confirm the endpoint is live (405 Method Not Allowed, not 404, with DELETE listed in the Allow header).

4. Send DELETE /identity/api/v2/admin/videos/{id} with the same regular user token.

5. Observe a 200 OK with a success message confirming the video was deleted, despite the account having no admin role.

Evidence

Day 8 testing — full request/response pairs and figures documented in Master Report Day 8 section.

Impact

Any authenticated user — regardless of privilege level — can destroy admin-managed content. In a production system this could be extended to other admin-only destructive actions (user bans, data deletion, configuration changes) depending on what else shares the same missing role check, making this a strong indicator of a systemic authorization gap rather than a one-off bug.

Risk Rating Justification

Rated Critical: this is a destructive, irreversible action (data deletion) performed with zero privilege verification, discoverable by anyone willing to read the API spec. No authentication trickery is needed — just a normal, valid low-privilege account.

F-04 — NoSQL Injection on Coupon Validation — Unauthorized Coupon Discovery

Severity: High

OWASP Category: API6 — Unrestricted Access to Sensitive Business Flows

Affected Endpoint(s): POST /community/api/v2/coupon/validate-coupon

Description

The coupon_code field in this endpoint's request body is passed into a MongoDB (NoSQL database) query without sanitization. Instead of sending a plain string, I sent NoSQL query operators — $ne (not equal) and $gt (greater than) — as the value of coupon_code. Because the field expected a string but received an object, the database ended up matching against the operator's logic instead of a literal code, which let me retrieve a valid, real coupon code that I had no prior knowledge of.

Steps to Reproduce

1. Send a normal POST /community/api/v2/coupon/validate-coupon request with a random string as coupon_code, and confirm it returns an 'invalid coupon' response.

2. Replace the coupon_code value with a NoSQL operator object, e.g. {"coupon_code":{"$ne":null}} or {"$gt":""}, instead of a plain string.

3. Send the request and observe the server returns a successful validation response containing a real, valid coupon code (e.g. TRAC075) that was never supplied or guessed.

4. Apply the extracted coupon code to confirm real business impact — observed account credit/balance increased by the coupon's value.

5. Additionally tested the $regex operator as a follow-up (Day 11): this crashed the endpoint with a 500 Internal Server Error rather than leaking data, confirming the injectable surface extends beyond the two operators that succeeded.

Evidence

Day 10 (original $ne / $gt bypass, confirmed balance impact) and Day 11 ($regex variation, 500 error). See Master Report Day 10 and Day 11 sections for full evidence including balance-before/after screenshots.

Impact

An attacker can discover and redeem valid coupon codes without ever needing to guess, phish, or purchase one, directly costing the business money through unearned discounts or credit. Since this is real, confirmed financial impact (not just a theoretical bypass), it represents a direct abuse-of-business-logic risk rather than a purely technical one.

Risk Rating Justification

Rated High rather than Critical because the impact is scoped to the coupon/credit system rather than full account or data compromise, and repeat-use of a single discovered code is correctly blocked (confirmed via re-application test — PASS). Still High because the discovery step requires no authorization and directly translates to real financial loss.

F-05 — Insecure Direct Object Reference — Mechanic Report Enumeration

Severity: High

OWASP Category: API1 — Broken Object Level Authorization (BOLA) / IDOR

Affected Endpoint(s): GET /workshop/api/mechanic/mechanic_report?report_id={id}

Description

This endpoint returns full mechanic service report details for a given report_id, but doesn't verify the requesting user actually owns the vehicle or service request tied to that report. Since report_id values are sequential integers, any report belonging to any user can be pulled just by incrementing the number.

Steps to Reproduce

1. As a legitimate user, submit a service request and note the report_id returned for your own report.

2. Send GET /workshop/api/mechanic/mechanic_report?report_id={different sequential number} that does not belong to your account.

3. Observe the full report details (problem description, VIN, mechanic assignment) for another user's report are returned with no ownership check.

4. (Day 11 padding) Confirm boundary handling: report_id=0, negative values, out-of-range values, and non-numeric strings are all rejected cleanly with appropriate error messages, ruling out crashes or type-confusion bypasses as an additional angle — the vulnerability is specifically the missing ownership check on valid, in-range IDs.

Evidence

Day 7 (original sequential enumeration finding) and Day 11 padding (boundary/type-confusion variations, all PASS). See Master Report Day 7 and Day 11 sections.

Impact

Any authenticated user can read every other user's service report history, exposing vehicle problem descriptions, VINs, and mechanic communications — a meaningful privacy exposure, though less severe than the live-location leak in F-02 since it doesn't expose real-time physical location.

Risk Rating Justification

Rated High: broad, low-effort horizontal data exposure across all users' service history, but scoped to service-report metadata rather than live tracking or account takeover, which is why it sits below F-02 despite being the same underlying vulnerability class.

F-06 — Server-Side Request Forgery via Mechanic Contact API

Severity: High

OWASP Category: API7 — Server-Side Request Forgery (SSRF)

Affected Endpoint(s): POST /workshop/api/merchant/contact_mechanic

Description

The mechanic_api field in this endpoint's request body is meant to hold a trusted, pre-configured mechanic service URL, but the server accepts and fetches whatever URL is supplied by the client, then reflects the result back in the response. This is a classic SSRF pattern — the server is making outbound requests on the attacker's behalf to a destination the attacker fully controls.

Steps to Reproduce

1. Send a normal POST /workshop/api/merchant/contact_mechanic request and confirm the mechanic_api field is present and accepted as a URL.

2. Replace the mechanic_api value with an attacker-controlled or internal-facing URL instead of the expected mechanic service endpoint.

3. Send the request and observe the server fetches the supplied URL and reflects its response content back to the client.

4. (Follow-up attempted, not completed) Intended to test reachability of internal-only addresses (localhost, Docker service names, cloud metadata endpoint 169.254.169.254) to establish blast radius, but ran out of time to get a clean result — noted as a gap for future testing rather than a confirmed sub-finding.

Evidence

Day 9 testing — request/response evidence documented in Master Report Day 9 section.

Impact

In this local lab environment the immediate impact is limited, but in a real cloud deployment, an unrestricted SSRF like this is a common path to reaching internal-only services, cloud metadata endpoints (which can expose cloud credentials), or other containers on the same Docker network that were never meant to be internet-reachable.

Risk Rating Justification

Rated High: the core vulnerability (server fetches arbitrary attacker-supplied URLs with no allow-list) is fully confirmed and directly exploitable. Not rated Critical here because full internal-network reachability (the piece that would typically escalate SSRF to Critical) wasn't conclusively demonstrated within this engagement's time constraints.

F-07 — Session Not Invalidated on Logout

Severity: Medium

OWASP Category: API2 — Broken Authentication

Affected Endpoint(s): Identity service — logout flow and subsequent authenticated requests using the same token

Description

Logging out through the normal application flow does not invalidate the JWT server-side. A token captured before logout continues to work against authenticated endpoints even after the user has logged out client-side.

Steps to Reproduce

1. Log in and capture a valid JWT.
2. Use the application's logout function.
3. Replay the previously captured token against an authenticated endpoint (e.g. GET /identity/api/v2/user/dashboard).
4. Observe the request still succeeds, confirming the token remains valid after logout.

Evidence

Day 6 testing — see Master Report Day 6 section.

Impact

If a token is ever captured (e.g. via a shared device, browser history, or a prior compromise), logging out does not actually revoke the attacker's access — they can continue using the stolen token until it naturally expires, regardless of the legitimate user's attempt to end the session.

Risk Rating Justification

Rated Medium: this is a real gap in session lifecycle management, but exploitation depends on the token already having been captured through some other means — it's not itself a way to obtain a token, only a way to keep using one that's already been compromised or shared.

F-08 — Security Misconfiguration — Open CORS Policy and Missing CSP

Severity: Medium

OWASP Category: API8 — Security Misconfiguration

Affected Endpoint(s): Application-wide — response headers across identity, workshop, and community services

Description

The application's CORS (Cross-Origin Resource Sharing) policy allows requests from any origin, and no Content Security Policy (CSP) header is set anywhere in the responses I reviewed. CORS controls which other websites are allowed to make authenticated requests to this API from a user's browser; CSP restricts what kinds of content (scripts, styles, frames) a page is allowed to load.

Steps to Reproduce

1. Review response headers across a range of endpoints (identity, workshop, community services) in Burp Suite.

2. Confirm the Access-Control-Allow-Origin header is set to * (wildcard, any origin) rather than a specific trusted domain list.

3. Confirm no Content-Security-Policy header is present in any response reviewed.

Evidence

Day 10 testing — see Master Report Day 10 section.

Impact

An open CORS policy widens the pool of malicious sites that could attempt to make authenticated cross-origin requests against this API using a logged-in victim's session. A missing CSP removes a layer of defense against content-injection attacks (like cross-site scripting) succeeding or executing effectively, should one occur elsewhere in the app.

Risk Rating Justification

Rated Medium: this is a defense-in-depth weakness rather than a directly exploitable vulnerability on its own — no working exploit chain through CORS or the missing CSP was demonstrated in this engagement — but both are widely recognized best-practice gaps that materially increase risk if combined with another flaw (e.g. an XSS finding elsewhere).

F-09 — Unsafe Consumption of Upstream API Responses

Severity: Medium

OWASP Category: API10 — Unsafe Consumption of APIs

Affected Endpoint(s): Various service-to-service calls, notably contact_mechanic response forwarding

Description

crAPI's backend services forward responses from upstream calls (including from attacker-influenced sources, as seen in the SSRF finding F-06) without validating them against an expected schema first. In some error conditions, raw internal exception text is passed straight through to the client instead of being caught and replaced with a generic error message.

Steps to Reproduce

1. Trigger error conditions on endpoints that make upstream/internal service calls (e.g. malformed input to contact_mechanic or similar forwarding endpoints).

2. Review the response body for raw exception text, stack traces, or internal implementation details rather than a clean, generic error message.

3. Confirm the response content varies directly based on what the upstream call returned, indicating no schema validation layer sits between the upstream response and the client-facing response.

Evidence

Day 10 testing — see Master Report Day 10 section.

Impact

Leaking internal exception text or unvalidated upstream content gives an attacker free reconnaissance into backend structure, library versions, or internal service names, which can inform further attacks (including making the SSRF finding in F-06 easier to exploit, since more of the internal environment becomes visible).

Risk Rating Justification

Rated Medium: an information-disclosure and defense-in-depth issue rather than a directly exploitable vulnerability on its own, but it compounds the risk of other findings (particularly F-06) by making the internal environment easier to map.

Risk Rating Methodology

I used a simple four-tier severity scale rather than a full CVSS (Common Vulnerability Scoring System) calculation, since that's proportionate to the scope of this engagement. Each rating below reflects a combination of how easy the issue is to exploit and how bad the consequences are if it's exploited.

Critical: Directly leads to full account takeover, unauthorized access to highly sensitive data (like live location), or destructive actions with no authorization check at all. Low effort to exploit, severe consequences.

High: Meaningful unauthorized access to data or business logic (e.g. financial impact, broad data exposure) but scoped more narrowly than a Critical finding, or requiring a slightly more specific technique to pull off.

Medium: Real weaknesses that increase risk or compound other findings, but don't by themselves grant unauthorized access or data exposure — often defense-in-depth gaps or session-hygiene issues.

Low: Minor issues, best-practice deviations, or hardening opportunities with limited standalone impact. (No findings in this engagement were rated Low.)

crAPI VAPT — API Security Checklist

OWASP API Security Top 10 (2023) — Pass/Fail Summary of everything I tested against crAPI (Completely Ridiculous API, local Docker instance).

Category	OWASP API Top 10 Category	What Was Tested	Result	Severity	Notes / Key Evidence
API1	Broken Object Level Authorization (BOLA)	Vehicle location endpoint (UUID swap across User A/B); mechanic report endpoint (sequential ID enumeration); plus edge-case variations (malformed, non-existent, empty IDs)	**FAIL**	Critical / High	I could swap IDs and pull another user's data on both endpoints — that's the core issue. The edge cases (malformed/empty/out-of-range IDs) were all handled safely, so it's specifically the ownership check that's missing, not input validation in general.
API2	Broken Authentication	JWT signature verification; session invalidation on logout; rate limiting on login and password-reset endpoints	**FAIL**	Critical / Medium	I was able to bypass JWT signature verification for full account impersonation (Critical), and logging out doesn't invalidate the session server-side (Medium). Login also has no rate limiting or lockout; logged separately under API4.
API3	Broken Object Property Level Authorization (Mass Assignment)	Phone-number-change endpoint; signup endpoint — tried injecting/overwriting protected fields (e.g. role) not exposed in the normal request	**PASS**	—	Couldn't get any unauthorized field to stick via mass assignment on either endpoint. Dashboard role stayed unchanged after every injection attempt.
API4	Unrestricted Resource Consumption	Rate limiting on login and password-reset endpoints; repeated OTP submission	**FAIL**	High	No limit on login or OTP-generation attempts: 15 and 20 requests accepted, no lockout or CAPTCHA. OTP-check 500 blocked brute-force testing there.
API5	Broken Function Level Authorization (BFLA)	Admin-only DELETE /identity/api/v2/admin/videos /{video_id} endpoint tested	**FAIL**	Critical	I was able to delete a video using a plain user account — no admin privileges required. I also logged this under API9, since I only found the

Category	OWASP API Top 10 Category	What Was Tested	Result	Severity	Notes / Key Evidence
		with a regular ROLE_USER token			endpoint by grepping the OpenAPI spec, not through normal app navigation.
API6	Unrestricted Access to Sensitive Business Flows	Coupon validation endpoint — NoSQL injection using $ne, $gt, and $regex operators in the coupon_code field	**FAIL**	High	$ne and $gt both bypassed validation completely and got me a valid coupon code I had no prior knowledge of, with a real balance increase to prove it. $regex also crashed the endpoint (500), so the injectable surface is bigger than just one operator.
API7	Server-Side Request Forgery (SSRF)	mechanic_api field on POST /workshop/api/merchant/contact_mechanic — supplied attacker-controlled URLs	**FAIL**	High	The server fetches whatever URL I put in the mechanic_api field and reflects it back, no allow-list or destination check at all. Confirms I could make the server send outbound requests on my behalf.
API8	Security Misconfiguration	CORS (Cross-Origin Resource Sharing) policy; Content Security Policy (CSP) header presence	**FAIL**	Medium	CORS is wide open (no origin restriction) and there's no CSP header at all. I didn't chain a direct exploit through these, but they widen the attack surface for cross-origin and content-injection attacks.
API9	Improper Inventory Management	v1 legacy endpoint checks across all three microservices (identity, workshop, community); OpenAPI spec review for undocumented endpoints	**FAIL**	Critical (shared w/ API5)	No exposed v1 legacy endpoints anywhere (that part's a clean pass). But grepping the OpenAPI spec turned up an undocumented admin video-delete endpoint I never would've found through the UI — a shadow API, same finding I recorded under API5 for the authorization impact.
API10	Unsafe Consumption of APIs	Upstream response handling and error propagation across service-to-service calls (e.g. contact_mechanic forwarding)	**FAIL**	Medium	crAPI forwards upstream responses without schema validation and leaks raw internal exception text in some error paths — gives away more about the backend structure than it should.

Category	OWASP API Top 10 Category	What Was Tested	Result	Severity	Notes / Key Evidence
GraphQL	GraphQL Attack Surface (supplementary check)	Searched for a GraphQL endpoint across crAPI and Juice Shop	**N/A**	—	Confirmed it's not there — crAPI is REST-only. Logged this as a Challenge item (with sign-off) rather than a coverage gap, since it's simply not part of the app's attack surface.

Summary

9 out of 10 OWASP API Top 10 categories failed at least one test on my end, spanning Critical, High, and Medium severity findings. 1 category (API3 — Mass Assignment) came back clean. I've got documented test evidence, screenshots, and reproduction steps for all 10 categories plus the GraphQL check in the full VAPT Report and Master Report.

crAPI VAPT — Remediation Tracker

Simple list of what I found, what should be fixed, and who owns fixing it. Everything's still Open right now since I just wrapped up testing.

ID	Finding	Recommended Fix	Owner	Severity	Status
F-01	JWT Signature Verification Bypass	Make the server actually check the token signature properly on every request, and reject anything that's been tampered with instead of trusting it. Add a test that tries replaying a modified token so this doesn't slip through again.	Backend / Identity Service Team	**Critical**	**Open**
F-02	BOLA — Cross-User Vehicle Location Access	Before returning a vehicle's location, check that the vehicle actually belongs to the person asking for it. Right now just knowing the ID is enough, which shouldn't be the case.	Backend / Identity Service Team	**Critical**	**Open**
F-03	BFLA — Unauthorized Admin Video Deletion	Add a proper admin-only check on this endpoint (and ideally all /admin/ routes at once) so a regular account can't call it. If it's not even supposed to be in use, just remove it.	Backend / Identity Service Team	**Critical**	**Open**
F-04	NoSQL Injection on Coupon Validation	Only accept a plain string for coupon_code, and reject anything else before it reaches the database. Basically stop trusting whatever shape of data the client sends.	Backend / Community Service Team	**High**	**Open**
F-05	IDOR — Mechanic Report Enumeration	Check that a report actually belongs to the person requesting it before returning it. Switching report IDs from simple counting numbers to something harder to guess would help too, but the ownership check is the real fix.	Backend / Workshop Service Team	**High**	**Open**
F-06	SSRF via Mechanic Contact API	Stop letting the client send any URL it wants in the mechanic_api field. Only allow a fixed list of approved mechanic URLs, or at least block internal/private addresses if it has to stay flexible.	Backend / Workshop Service Team	**High**	**Open**

ID	Finding	Recommended Fix	Owner	Severity	Status
F-07	Session Not Invalidated on Logout	Make logout actually kill the token server-side instead of just clearing it on the frontend, so an old token can't keep being used afterward.	Backend / Identity Service Team	**Medium**	**Open**
F-08	Open CORS Policy and Missing CSP	Lock CORS down to only the actual frontend domain instead of allowing everything. Also add a CSP header, which isn't set anywhere right now.	Platform / DevOps Team	**Medium**	**Open**
F-09	Unsafe Consumption of Upstream API Responses	Stop passing raw internal error messages straight to the client — swap them for a generic error instead. Also check that responses from other internal calls actually look the way they're supposed to before forwarding them.	Backend / Platform Team	**Medium**	**Open**

How to Use This

As things get fixed, just update the Status column: Open → In Progress → Fixed → Verified. I'd recommend actually re-testing each one the same way I originally broke it before marking it Verified, rather than assuming a code change fixed it.

I'd fix the 3 Critical ones first (F-01, F-02, F-03) since they let someone take over an account, see someone else's live location, or delete stuff they shouldn't be able to touch. F-01 especially — if someone can still fake a login token, fixing the other access checks won't fully help since they can just get around them the same way.